ATTENDANCE AUTOMATION SYSTEM USING QR CODE, GPS AND CAMERA VERIFICATION

Name:

Tehreem Shakir

University:

Air University, Islamabad

Department:

Computer Engineering

Internship Organization:

SafeX Solutions

Date:

18th July, 2026

Contents

1. Project Planning & Requirement Analysis

1.1 Problem Statement

Many organizations still use manual attendance systems such as paper registers or spreadsheets to record employee attendance. These methods are time-consuming, prone to human errors, and can be manipulated through proxy attendance. Managing attendance records manually also increases the workload for administrators and makes report generation difficult.

This project aims to automate the attendance process by using modern technologies such as QR Code verification, GPS location verification, camera verification, and a centralized database. The system improves accuracy, security, and efficiency while reducing manual work.

1.2 Objectives

The main objectives of the Attendance Automation System are:

- Automate the employee attendance process.
- Provide secure employee login.
- Use a QR Code to initiate the attendance process.
- Verify the employee's location using GPS.
- Capture a selfie using the device's camera during attendance.
- Prevent duplicate attendance records.
- Store attendance records in a centralized database.
- Detect late arrivals automatically.
- Generate attendance reports in Excel and PDF formats.
- Provide an administrator dashboard for managing employees and attendance records.

1.3 Users

Employee

The employee can:

- Login securely.
- Generate and verify the QR Code.
- Allow GPS location access.
- Allow camera access and capture a selfie.
- Mark attendance.

- View attendance history.
- Logout securely.

Administrator

The administrator can:

- Login securely.
- View the Admin Dashboard.
- Add new employees.
- Delete employee records.
- View all attendance records.
- Search attendance by employee username.
- Filter attendance records by date.
- Monitor late employees.
- Export attendance reports to Excel.
- Export attendance reports to PDF.

1.4 Features

Employee Features

- Secure Login
- QR Code Attendance Verification
- GPS Location Verification
- Camera Selfie Verification
- Duplicate Attendance Prevention
- Attendance History
- Secure Logout

Administrator Features

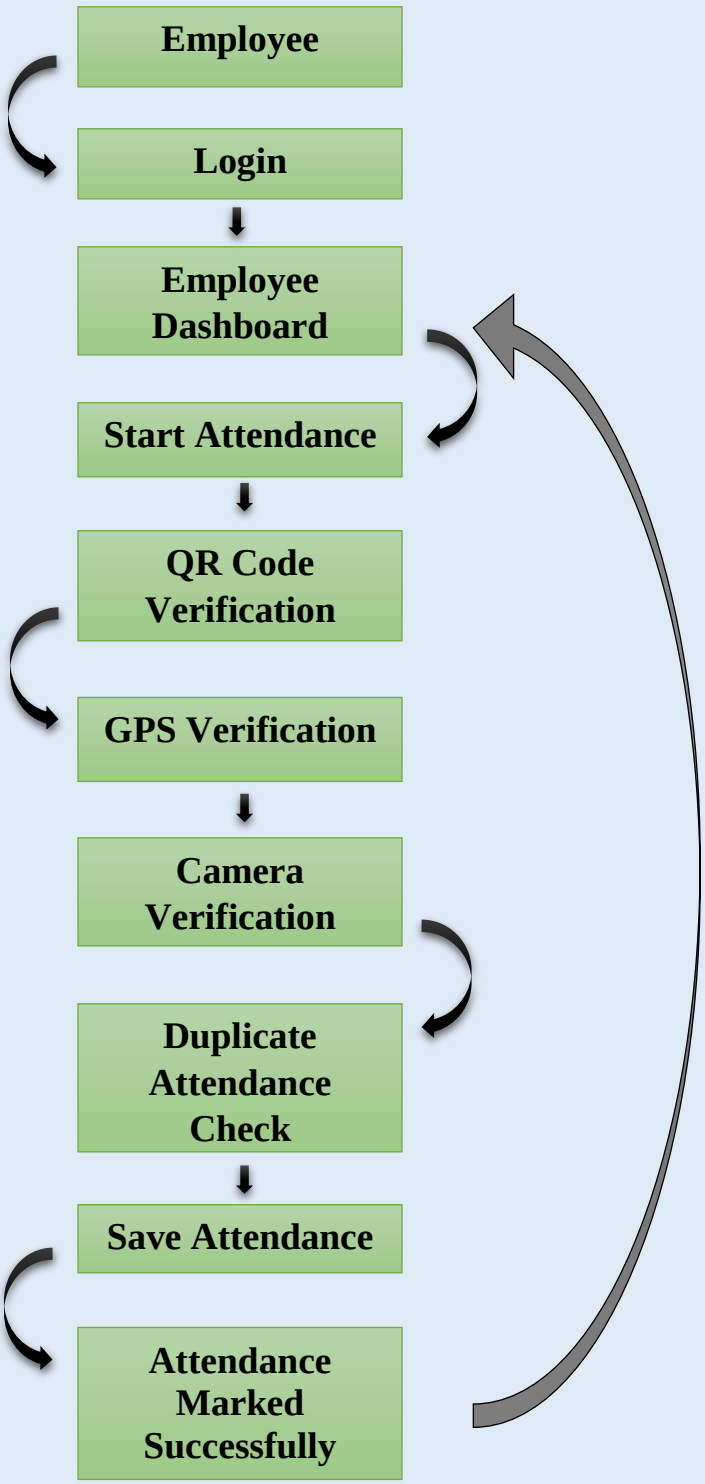
- Employee Management
- Admin Dashboard
- Attendance Record Management
- Search Attendance Records
- Date Filter
- Excel Report Export
- PDF Report Export
- Late Attendance Monitoring

1.5 Technologies Used

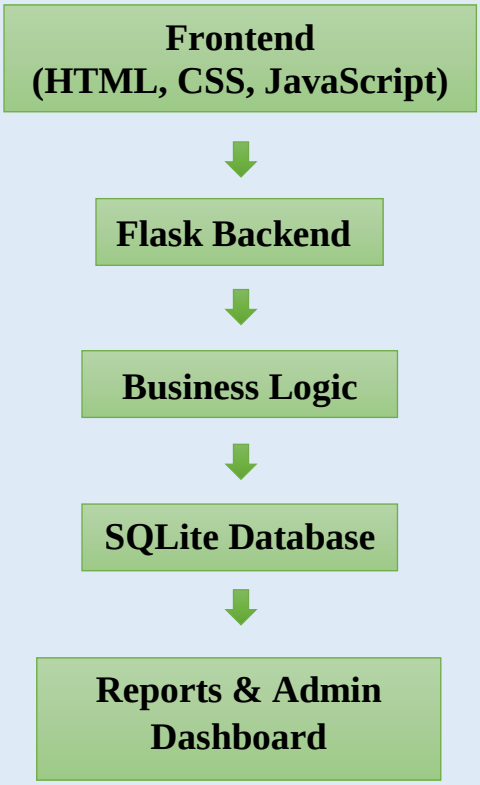
Technology	Purpose
Python	Main programming language
Flask	Backend web framework
HTML5	Structure of web pages
CSS3	Styling and user interface
JavaScript	GPS and camera access, client-side interactions
SQLite	Database management

qrcode	QR Code generation
OpenPyXL	Excel report generation
ReportLab	PDF report generation
Font Awesome	Icons used in the user interface
Google Fonts (Poppins)	Modern typography for the web interface

1.6 System Workflow

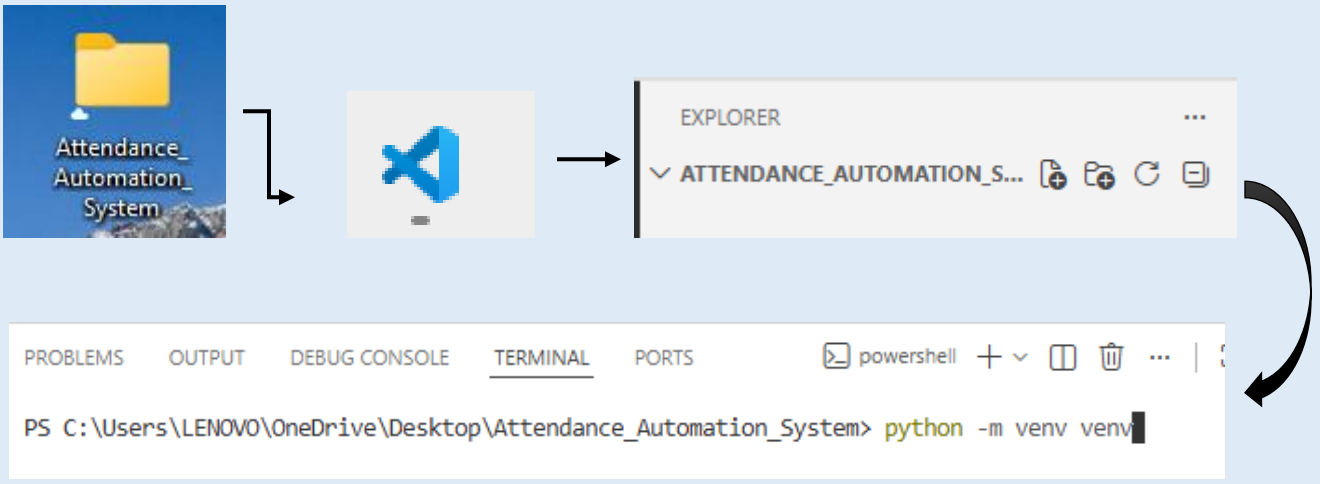


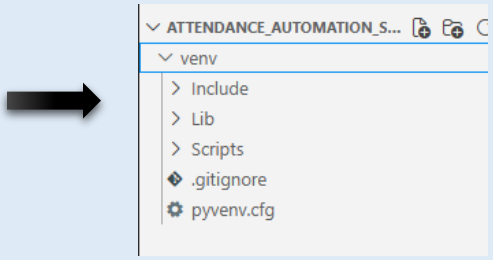
1.7 Project Architecture



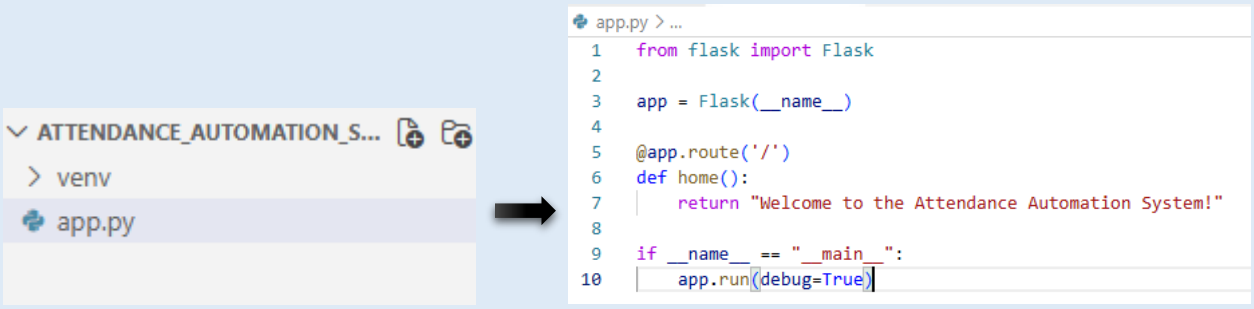
2. Environment Setup

- Project Folder Creation
- Opening the Project in VS Code
- Creating a Virtual Environment
- Activating the Virtual Environment
- Installing Required Libraries

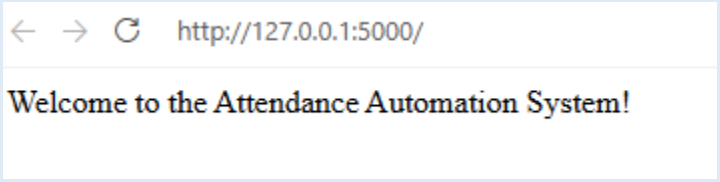




- New file created:



- Run the Flask Application:

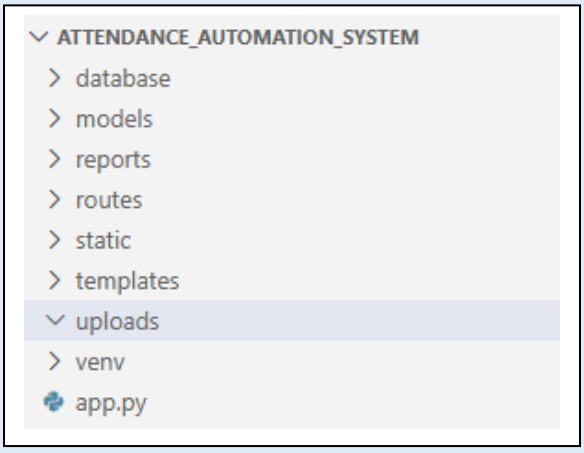


The Flask application was executed using the Python interpreter. After starting the development server, the application was successfully accessed through a web browser using the local host address (http://127.0.0.1:5000). The successful display of the welcome message confirmed that the development environment was correctly configured.

3. Project Development

3.1 Create the Flask Project Structure

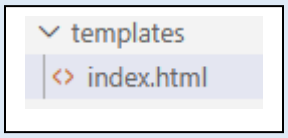
- Created separate folders for **templates**, **static**, **database**, **reports**, and other project resources.
- Organized the project using a modular folder structure.
- Improved code management and future maintenance.



- Followed the standard Flask project structure.

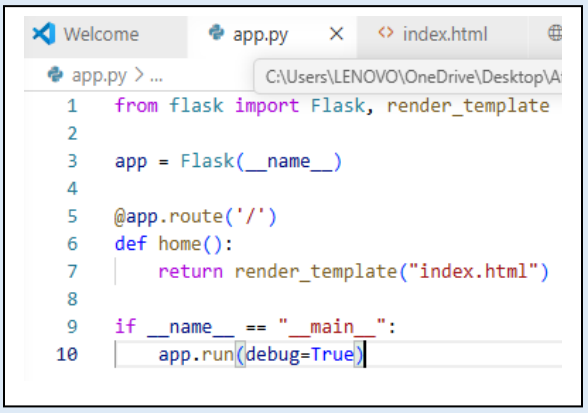
3.2 Create the First HTML Page

- Created **index.html** inside the **templates** folder.
- Designed it as the home page of the Attendance Automation System.
- Used the templates folder to allow Flask to render HTML pages.



3.3 Display the HTML Page Using Flask

- Updated the Flask application to use the **render_template()** function.
- Connected the backend with the frontend.
- Successfully displayed **index.html** in the web browser.

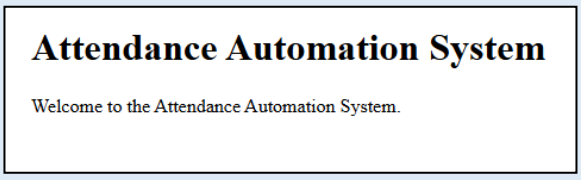


3.4 Add Content to the Home Page

- Added the basic HTML5 page structure.
- Included a page title and welcome message.
- Verified that the page loaded correctly through Flask.

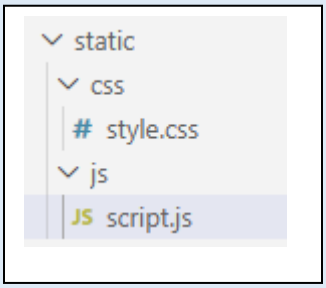
templates > index.html > html

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0
6      <title>Attendance Automation System</title>
7  </head>
8  <body>
9
10     <h1>Attendance Automation System</h1>
11
12     <p>Welcome to the Attendance Automation System.</p>
13
14 </body>
15 </html>
```



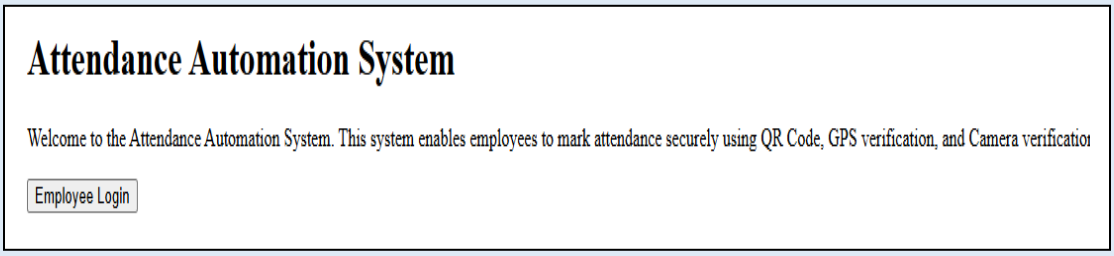
3.5 Create Static Resource Files

- Created separate **CSS** and **JavaScript** folders inside the **static** directory.
- Added **style.css** and **script.js** files.
- Organized styling and client-side functionality.



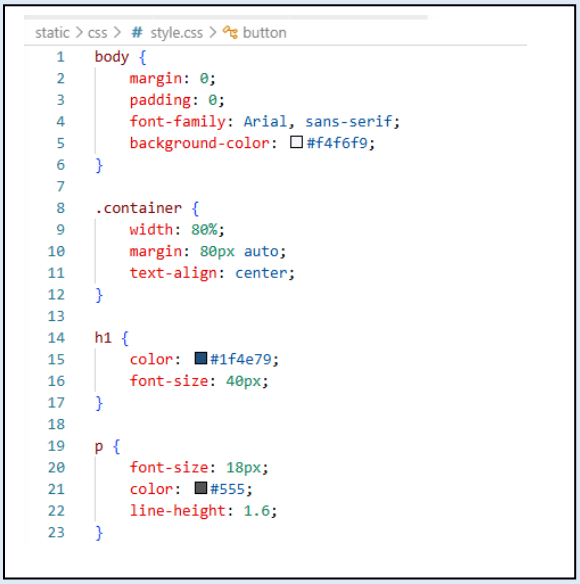
3.6 Update the Home Page

- Added a system title and welcome message.
- Created an **Employee Login** button.
- Linked external CSS and JavaScript files.
- Improved the page layout.

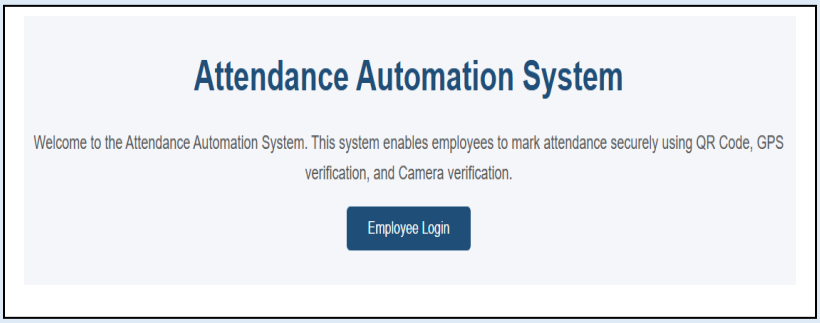


3.7 Style the Home Page

- Designed a clean and professional interface using CSS.
- Applied styling to the background, text, and buttons.
- Improved the overall user experience.



```
static > css > # style.css > button
1   body {
2       margin: 0;
3       padding: 0;
4       font-family: Arial, sans-serif;
5       background-color: #f4f6f9;
6   }
7
8   .container {
9       width: 80%;
10      margin: 80px auto;
11      text-align: center;
12  }
13
14  h1 {
15      color: #1f4e79;
16      font-size: 40px;
17  }
18
19  p {
20      font-size: 18px;
21      color: #555;
22      line-height: 1.6;
23  }
```

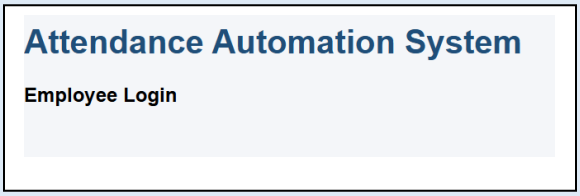


3.8 Create the Login Page

- Created **login.html** inside the **templates** folder.
- Designed it as the employee authentication page.
- Connected it with the shared CSS stylesheet.

templates > login.html > html

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Employee Login</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>

<body>

</body>

</html>
```



3.9 Create the Login Card

- Designed a centered login card.
- Added the system title and login heading.
- Improved the layout and appearance of the login page.

static > css > # style.css > .login-card h2

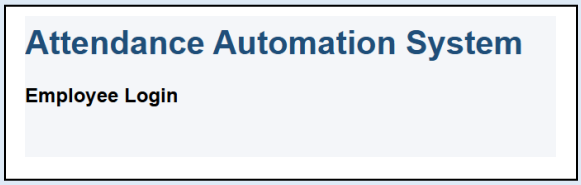
```
button:hover {
}
/* Login Page */

.login-card {
    width: 400px;
    margin: 80px auto;
    background-color: white;
    padding: 40px;
    border-radius: 10px;
    box-shadow: 0px 4px 15px rgba(0, 0, 0, 0.2);
    text-align: center;
}

.login-card h1 {
    color: #1f4e79;
    margin-bottom: 10px;
}

.login-card h2 {
    color: #555;
    margin-bottom: 30px;
}
```

3.10 Create the Login Route

- Created the **/login** route in Flask.
- Configured the route to display **login.html**.
- Enabled navigation to the login page.

app.py > ...

```python
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
    return render_template("index.html")


@app.route('/login')
def login():
    return render_template("login.html")


if __name__ == "__main__":
    app.run(debug=True)
```



3.11 Style the Login Page

- Applied CSS styling to the login card.
- Added rounded corners, spacing, and shadow effects.
- Created a modern and responsive interface.

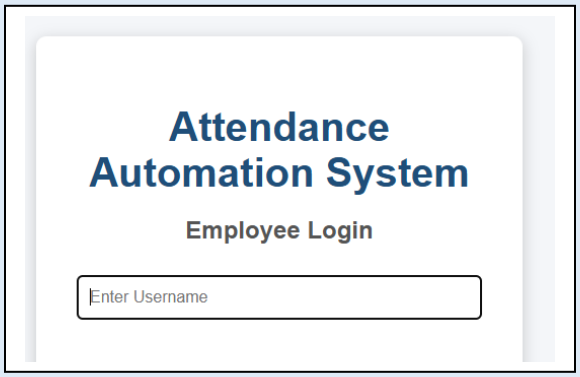
static > css > # style.css > .login-card h2

```css
button:hover {
}
/* Login Page */

.login-card {
    width: 400px;
    margin: 80px auto;
    background-color: white;
    padding: 40px;
    border-radius: 10px;
    box-shadow: 0px 4px 15px rgba(0, 0, 0, 0.2);
    text-align: center;
}

.login-card h1 {
    color: #1f4e79;
    margin-bottom: 10px;
}

.login-card h2 {
    color: #555;
    margin-bottom: 30px;
}
```

3.12 Add the Username Field

- Added a username input field.
- Applied consistent styling using CSS.
- Improved the usability of the login form.

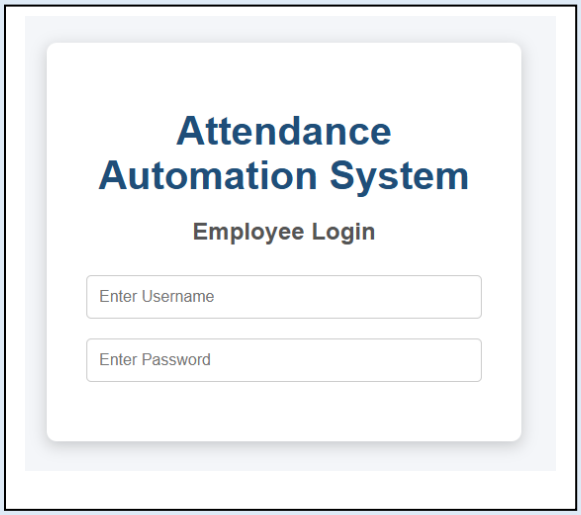
```
<body>
    <div class="login-card">
        <h1>Attendance Automation System</h1>
        <h2>Employee Login</h2>
        <input type="text" placeholder="Enter Username">
    </div>
</body>
```



3.13 Add the Password Field

- Added a password input field.
- Used the **password** input type for secure data entry.
- Maintained a consistent page design.

```
        <h2>Employee Login</h2>
        <input type="text" placeholder="Enter Username">
        <input type="password" placeholder="Enter Password">
    </div>
</body>
```

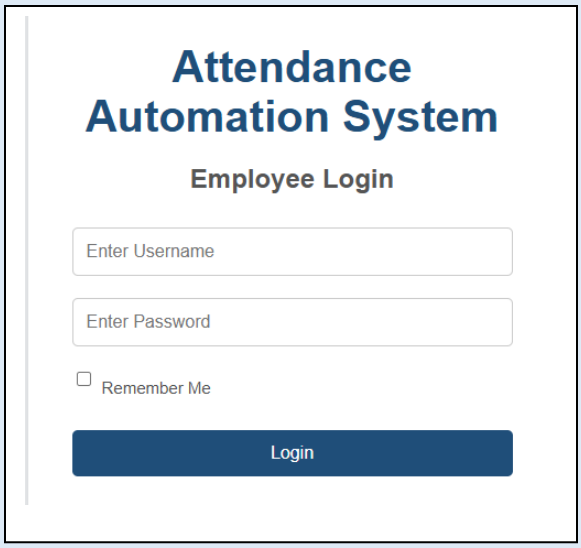


3.14 Add the Login Button

- Added a Login button below the input fields.
- Applied custom CSS styling.
- Included hover effects for better user interaction.
- Prepared the button for backend authentication.

```
        <label for="remember">Remember me</label>
    </div>
    <button class="login-btn">Login</button>
</div>
```

```css
.login-btn {
    width: 100%;
    background-color: #1f4e79;
    color: white;
    border: none;
    padding: 12px;
    font-size: 16px;
    border-radius: 5px;
    cursor: pointer;
    transition: background-color 0.3s ease;
}

.login-btn:hover {
    background-color: #163d5c;
}
```



4. Backend Development

4.1 Understanding HTML Forms

- Studied the purpose of HTML forms for collecting user input.
- Used the login form to collect the employee's username and password.
- Configured the form to send data securely to the Flask backend using the **POST** method.
- Established communication between the frontend and backend.

4.2 Creating the Login Form

- Enclosed the username and password fields inside an HTML **<form>** element.
- Configured the form to submit data to the **/login** route.
- Used the **POST** method to send login credentials securely.
- Added a **Remember Me** checkbox and a **Login** button.

```html
    <div class="login-card">

        <h1>Attendance Automation System</h1>

        <h2>Employee Login</h2>

        <form action="/login" method="POST">

            <input type="text" placeholder="Enter Username">

            <input type="password" placeholder="Enter Password">

            <div class="remember-me">
                <input type="checkbox" id="remember">
                <label for="remember">Remember Me</label>
            </div>

            <button class="login-btn">Login</button>

        </form>

    </div>

</body>
```

4.3 Receiving Login Data in Flask

- Updated the **/login** route to handle both **GET** and **POST** requests.
- Used Flask's **request** object to receive form data.
- Retrieved the username and password using **request.form**.
- Displayed the submitted values in the terminal for testing.

app.py > ...

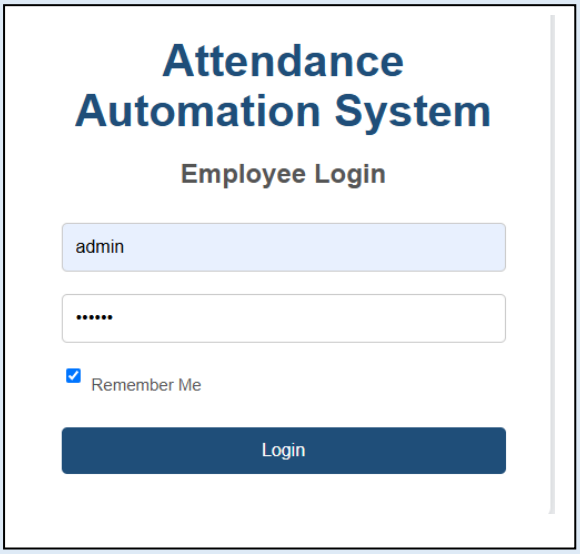
```python
from flask import Flask, render_template, request

app = Flask(__name__)

```

```python
@app.route('/login', methods=['GET', 'POST'])
def login():

    if request.method == 'POST':

        username = request.form['username']
        password = request.form['password']

        print("Username:", username)
        print("Password:", password)

    return render_template("login.html")
```

```html
<input type="text" name="username" placeholder="Enter Username">

<input type="password" name="password" placeholder="Enter Password">
```

4.4 Testing Form Submission

- Entered sample login credentials through the login page.
- Submitted the form to the Flask backend.
- Verified that the submitted data was successfully received.
- Confirmed proper communication between the frontend and backend.



5. Database Development

5.1 Understanding the Database

- Studied the role of databases in storing and managing application data.
- Selected **SQLite** as the database because it is lightweight and fully compatible with Python.
- Planned to store employee information, login credentials, and attendance records.
- Used the database to support user authentication and attendance management.

5.2 Creating the SQLite Database

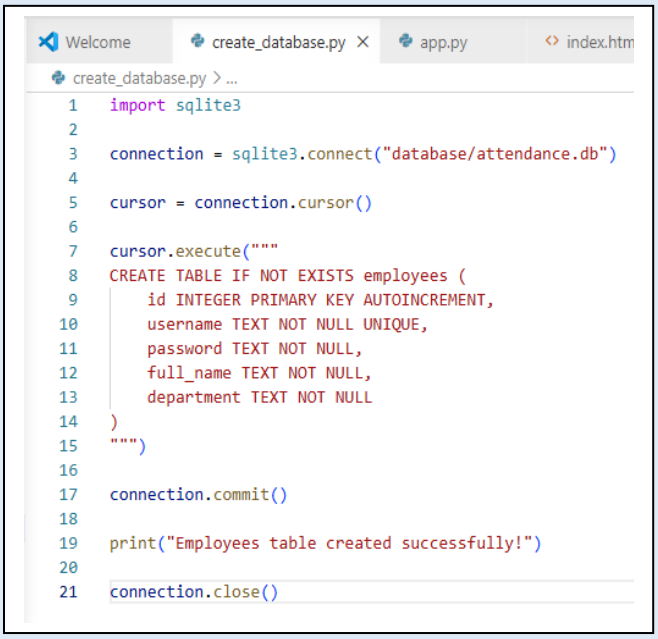
- Created a Python script named **create_database.py**.
- Used Python's **sqlite3** library to establish a database connection.
- Created the **attendance.db** file inside the **database** folder.
- Closed the database connection after successful creation.

5.3 Understanding Database Tables

- Learned how database tables organize data into rows and columns.
- Planned the **Employees** table to store employee information.
- Included fields for employee ID, username, password, full name, and department.
- Assigned a unique primary key to identify each employee.

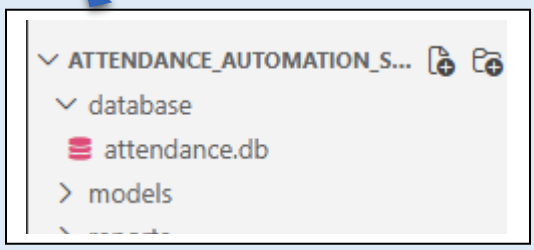
5.4 Creating the Employees Table

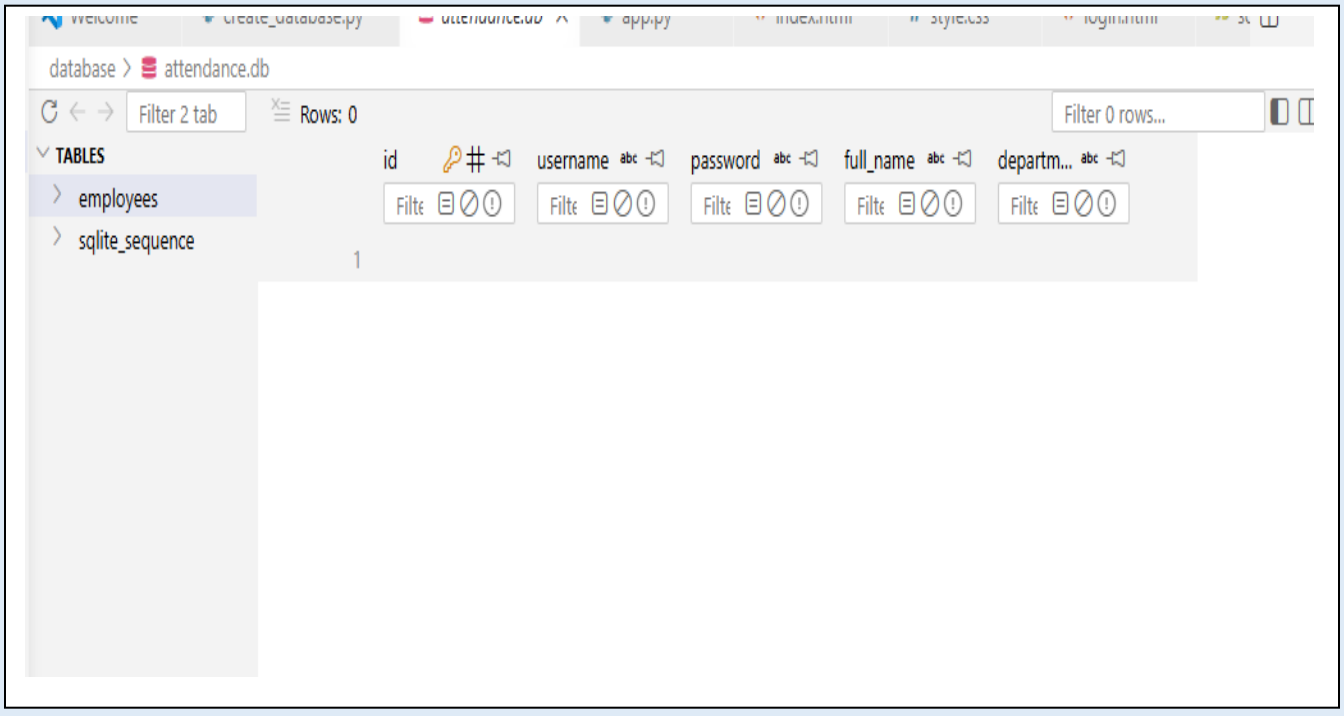
- Created the **Employees** table using an SQL **CREATE TABLE** statement.
- Added fields for employee ID, username, password, full name, and department.
- Applied **PRIMARY KEY**, **AUTOINCREMENT**, **NOT NULL**, and **UNIQUE** constraints.
- Saved the table using the **commit()** method.



5.5 Verifying the Employees Table

- Opened the SQLite database using the **SQLite Viewer** extension in Visual Studio Code.
- Verified that the **Employees** table was created successfully.
- Confirmed that the table structure matched the project requirements.
- Verified that the table was initially empty before inserting records.





5.6 Inserting the First Employee Record

- Inserted the first employee into the **Employees** table.
- Used an SQL **INSERT** statement to store employee information.
- Verified that the employee record was successfully added.
- Confirmed that the database was functioning correctly.

```
Welcome | create_database.py | attendance.db | app.py
create_database.py > ...
15  """)
16
17  cursor.execute("""
18  INSERT INTO employees (username, password, full_name, department)
19  VALUES (?, ?, ?, ?)
20  """, ("admin", "12345", "Admin User", "HR"))
21
22  connection.commit()
```

	id	username	password	full_name	departm...
	Filte	Filte	Filte	Filte	Filte
1	1	admin	12345	Admin User	HR

TEXT

5.7 Separating Database Creation and Employee Insertion

- Separated database creation and employee insertion into different Python files.
- Used **create_database.py** to create the database and tables.
- Used **add_employee.py** to insert employee records.
- Improved the project's organization, readability, and maintainability.

```
create_database.py > ...
1   import sqlite3
2
3   connection = sqlite3.connect("database/attendance.db")
4
5   cursor = connection.cursor()
6
7   cursor.execute("""
8   CREATE TABLE IF NOT EXISTS employees (
9       id INTEGER PRIMARY KEY AUTOINCREMENT,
10      username TEXT NOT NULL UNIQUE,
11      password TEXT NOT NULL,
12      full_name TEXT NOT NULL,
13      department TEXT NOT NULL
14  )
15  """)
16
17  connection.commit()
18
19  print("Database and Employees table created successfully!")
20
21  connection.close()
```

```
add_employee.py > ...
1   import sqlite3
2
3   connection = sqlite3.connect("database/attendance.db")
4
5   cursor = connection.cursor()
6
7   cursor.execute("""
8   INSERT INTO employees (username, password, full_name, department)
9   VALUES (?, ?, ?, ?)
10  """, ("admin", "12345", "Admin User", "HR"))
11
12  connection.commit()
13
14  print("Employee added successfully!")
15
16  connection.close()
```

6. Login Authentication

6.1 Understanding Login Authentication

- Studied the purpose of user authentication in web applications.
- Used the login system to verify employee credentials.
- Compared the entered username and password with records stored in the SQLite database.
- Allowed access only to users with valid credentials.

6.2 Connecting Flask with the Database

- Imported the **sqlite3** library into the Flask application.
- Established a connection between Flask and the SQLite database.
- Reviewed the existing login function to understand how user input was received.
- Prepared the application for database-based authentication.

```python
from flask import Flask, render_template, request
import sqlite3
app = Flask(__name__)

@app.route('/login', methods=['GET', 'POST'])
def login():

    if request.method == 'POST':

        username = request.form['username']
        password = request.form['password']

        print("Username:", username)
        print("Password:", password)

    return render_template("login.html")

if __name__ == "__main__":
    app.run(debug=True)
```

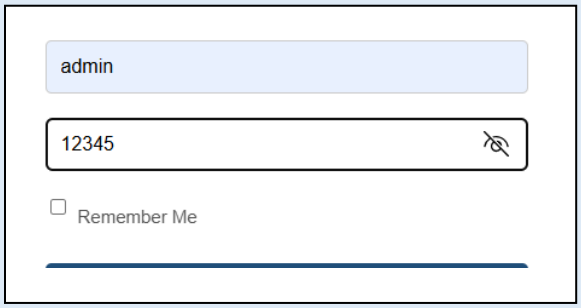
6.3 Login Function Before Database Authentication

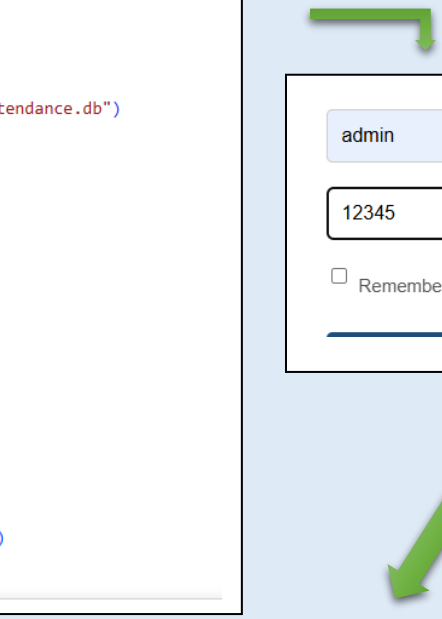
- Received the username and password from the login form.
- Retrieved user input using **request.form**.
- Displayed the entered credentials in the terminal for testing.
- Verified successful communication between the frontend and backend.

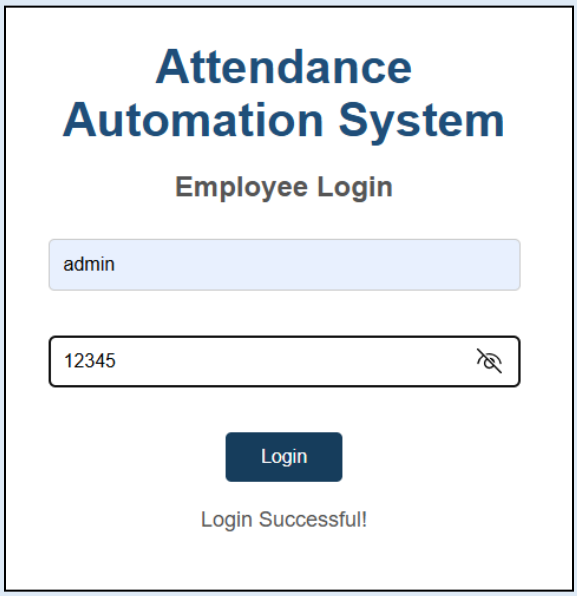
6.4 Implementing Database Authentication

- Connected the login functionality with the SQLite database.
- Executed an SQL **SELECT** query to search for matching user credentials.
- Used the **fetchone()** method to retrieve the matching employee record.
- Determined whether the login attempt was successful or invalid based on the query result.

```python
if request.method == 'POST':

    username = request.form['username']
    password = request.form['password']

    connection = sqlite3.connect("database/attendance.db")

    cursor = connection.cursor()

    cursor.execute("""
    SELECT * FROM employees
    WHERE username = ?
    AND password = ?
    """, (username, password))

    employee = cursor.fetchone()

    connection.close()

    if employee:
        print("Login Successful!")

    else:
        print("Invalid Username or Password!")

return render_template("login.html")
```

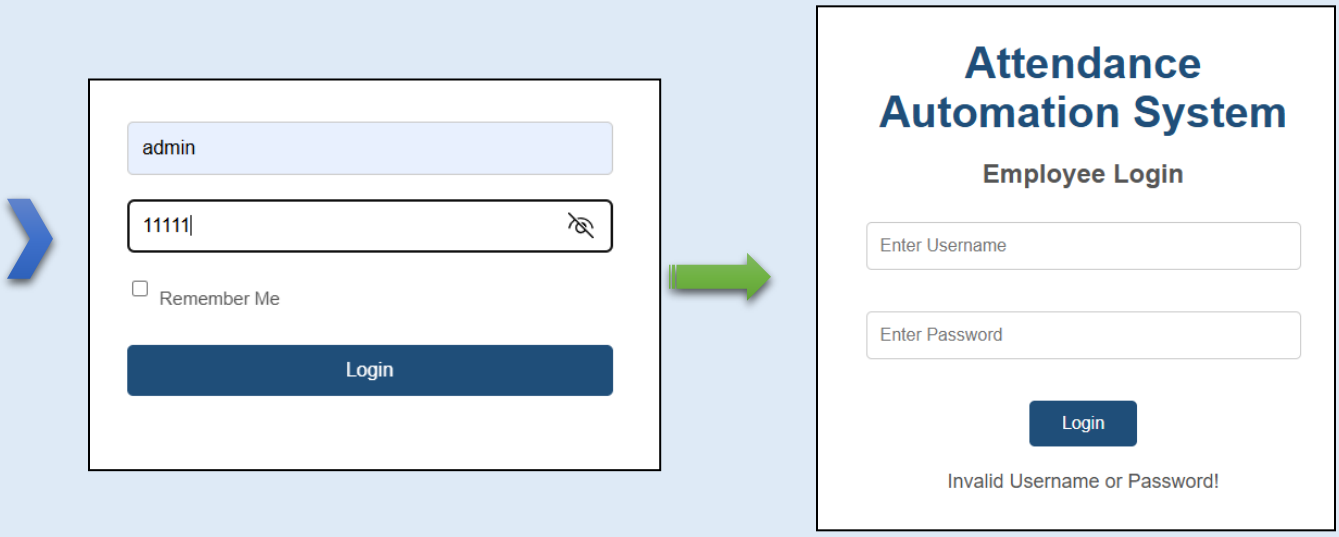






6.5 Displaying Authentication Results

- Replaced terminal output with user-friendly messages on the login page.
- Used a **message** variable to store authentication results.
- Passed the message to the HTML template using **render_template()**.
- Displayed appropriate success or error messages after each login attempt.



7. Employee Dashboard & Session Management

7.1 Understanding the Employee Dashboard

- Studied the purpose of an employee dashboard after successful login.
- Designed the dashboard as the main interface of the Attendance Automation System.
- Planned to provide quick access to attendance-related features.
- Improved usability by offering a centralized workspace for employees.

7.2 Creating the Employee Dashboard Page

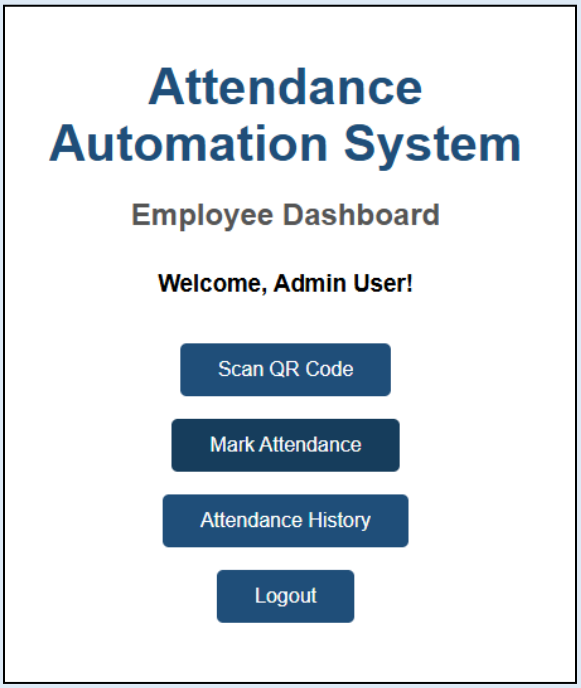
- Created a new HTML file named **dashboard.html**.
- Designed the basic dashboard layout.
- Added a welcome message for authenticated users.
- Included buttons for attendance, attendance history, and logout.

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Employee Dashboard</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">

</head>
```

7.3 Connecting the Dashboard with Flask

- Created a new Flask route for the employee dashboard.
- Used **redirect()** and **url_for()** to navigate users after successful login.
- Redirected authenticated users from the login page to the dashboard.
- Improved the overall user experience after authentication.

```python
@app.route('/dashboard')
def dashboard():
    return render_template("dashboard.html")


# Check login result
if employee:
    return redirect(url_for('dashboard'))
else:
    message = "Invalid Username or Password!"
```

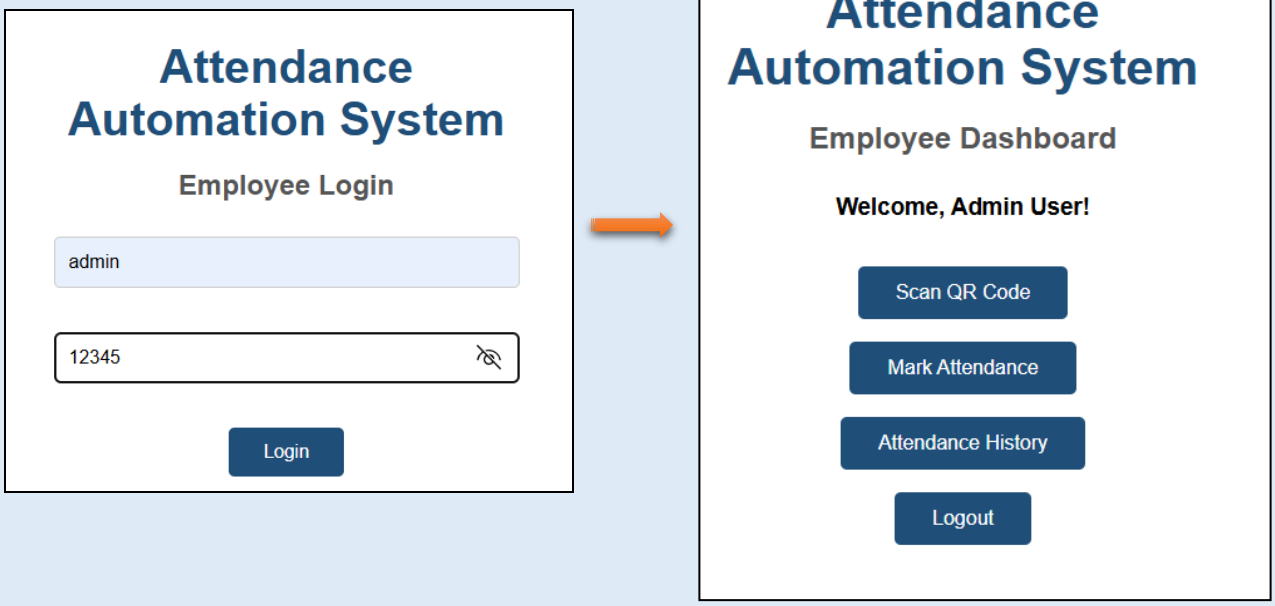


7.4 Understanding Flask Sessions

- Studied the purpose of sessions in web applications.
- Used sessions to remember authenticated users.
- Prevented unauthorized access to protected pages.
- Planned to remove sessions during logout.

7.5 Creating a Login Session

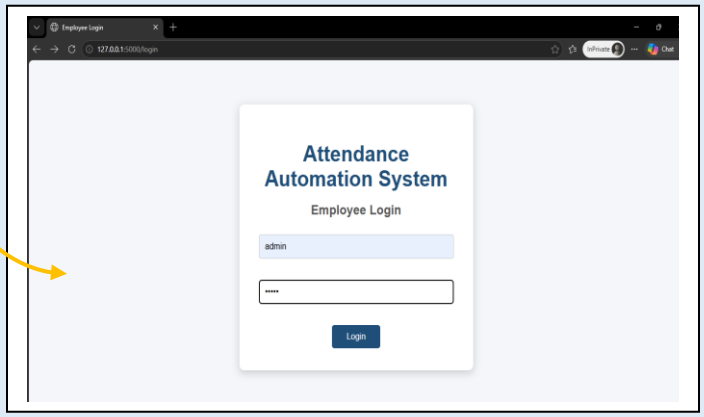
- Configured a secret key for secure session management.
- Stored the authenticated user's username in the session.
- Maintained the user's login state while navigating the application.
- Improved security by tracking authenticated users.



7.6 Protecting the Dashboard Using Sessions

- Checked for a valid user session before displaying the dashboard.
- Redirected unauthenticated users to the login page.
- Restricted dashboard access to logged-in users only.
- Improved the security of the Attendance Automation System.

```python
@app.route('/dashboard')
def dashboard():

    if 'username' not in session:
        return redirect(url_for('login'))

    return render_template("dashboard.html")


if __name__ == "__main__":
    app.run(debug=True)
```



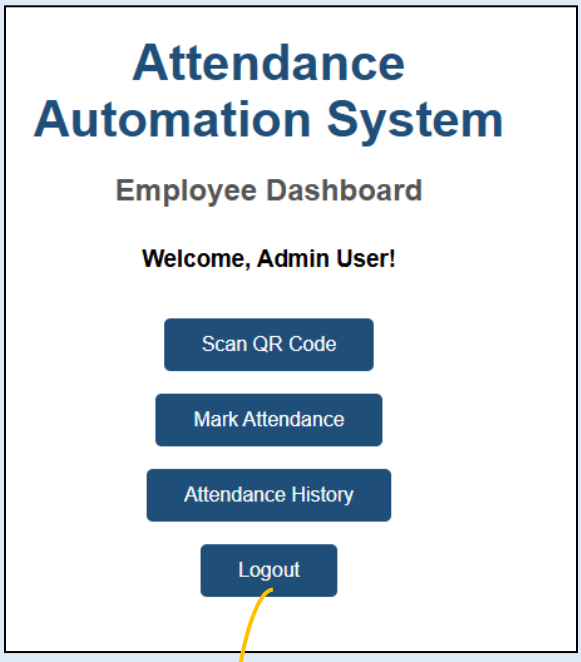
7.7 Implementing Logout Functionality

- Created a dedicated **/logout** route in Flask.
- Removed the user's session during logout.
- Redirected the user back to the login page.
- Prevented access to protected pages after logout.

```python
@app.route('/logout')
def logout():

    session.pop('username', None)

    return redirect(url_for('login'))
if __name__ == "__main__":
```

7.8 Logout Route Using Flask Sessions

- Used **session.pop()** to remove stored session data.
- Cleared the user's authentication information.
- Ended the current login session securely.

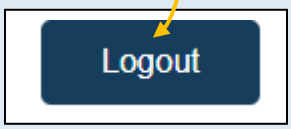


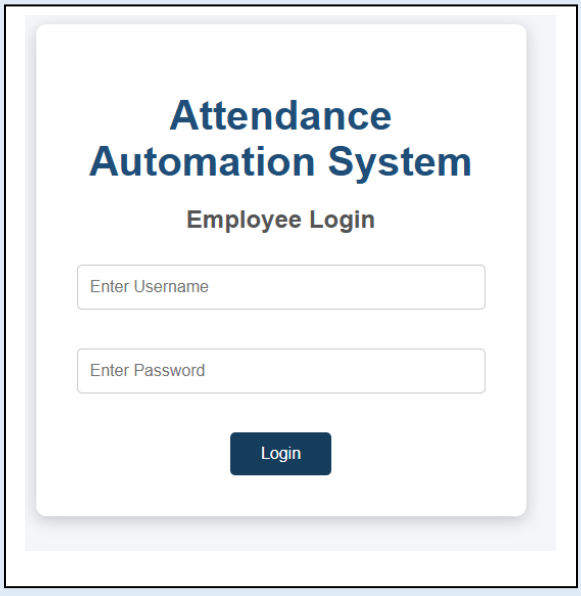
7.9 Connecting the Logout Button

- Linked the Logout button to the **/logout** route.
- Allowed users to securely exit the system.
- Ensured smooth navigation back to the login page.

7.10 Verifying Logout Functionality

- Confirmed that users were redirected to the login page after logout.
- Verified that the dashboard could no longer be accessed without logging in again.
- Ensured that session-based security was functioning correctly.





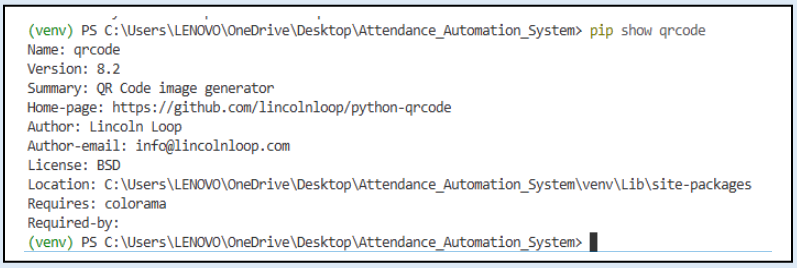
8. QR Code Attendance

8.1 Understanding QR Code Attendance

- Studied the use of QR codes for automating attendance.
- Used the QR code as the first step of the attendance process.
- Reduced the need for manual attendance marking.
- Prepared the workflow for GPS and camera verification.

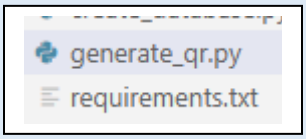
8.2 Installing the QR Code Library

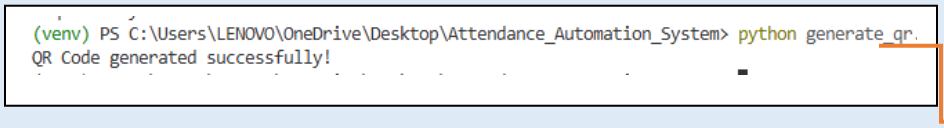
- Installed the **qrcode** library and **Pillow** package.
- Verified the installation using the **pip show** command.
- Updated the project requirements file.
- Prepared the application for QR code generation.

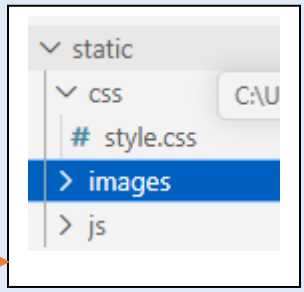


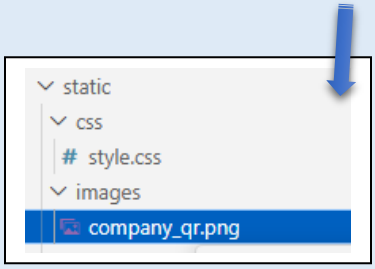
8.3 Generating the Company QR Code

- Created a Python script named **generate_qr.py**.
- Generated a QR code containing the application URL.
- Saved the QR code as **company_qr.png**.
- Stored the image inside the **static/images** folder.









8.4 Displaying the QR Code in the Web Application

- Created a new page named **qr_code.html**.
- Added a Flask route to display the QR code page.
- Connected the **Start Attendance** button to the QR verification page.
- Displayed the QR code from the **static/images** directory.

9. GPS Verification

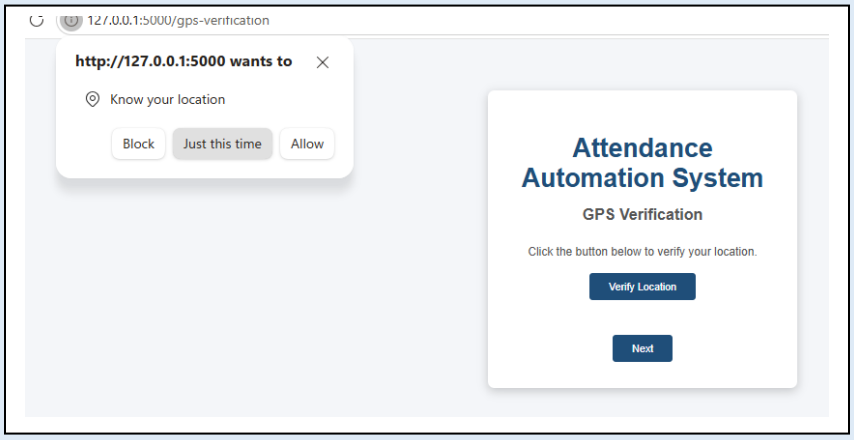
9.1 Understanding GPS Verification

```python
@app.route('/gps-verification')
def gps_verification():

    # Protect GPS Verification page
    if 'username' not in session:
        return redirect(url_for('login'))

    return render_template("gps_verification.html")
```

- Added GPS verification as the second step of attendance.
- Used the browser's **Geolocation API** to request the employee's location.
- Verified location before allowing attendance to continue.
- Added an extra layer of security to the attendance process.

9.2 Implementing GPS Verification

- Created a dedicated GPS verification section.
- Requested location permission from the user.
- Allowed attendance to continue only after successful location verification.
- Displayed a confirmation message after successful verification.

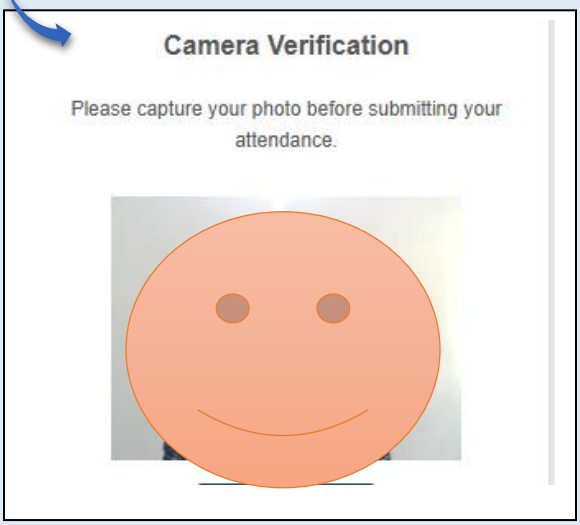


10. Camera Verification

10.1 Understanding Camera Verification

- Added camera verification as the final verification step.
- Required employees to verify their identity before attendance was recorded.
- Improved attendance security using webcam access.
- Prevented unauthorized attendance marking.

```python
@app.route('/camera-verification')
def camera_verification():

    # Protect Camera Verification page
    if 'username' not in session:
        return redirect(url_for('login'))

    return render_template("camera_verification.html")
```



10.2 Implementing Camera Verification

- Created the camera verification interface.
- Used the browser's **getUserMedia()** API to access the webcam.
- Allowed employees to capture a photo.
- Completed the verification process before recording attendance.

11. Attendance Recording

11.1 Recording Employee Attendance

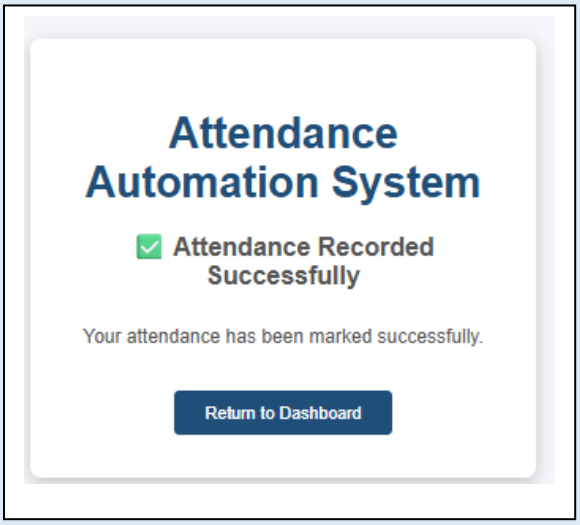
- Recorded attendance after successful QR, GPS, and camera verification.
- Stored attendance information in the SQLite database.
- Saved the employee username, date, time, and attendance status.
- Prevented duplicate attendance records for the same day.

templates > attendance_success.html > html > head

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Attendance Successful</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">

</head>

<body>

    <div class="login-card">

        <h1>Attendance Automation System</h1>

        <h2>✅ Attendance Recorded Successfully</h2>

        <p>Your attendance has been marked successfully.</p>

        <br>

        <a href="{{ url_for('dashboard') }}">
            <button>Return to Dashboard</button>
        </a>

    </div>
</body>
</html>
```

11.2 Attendance Confirmation

- Displayed a confirmation message after attendance was recorded.
- Redirected the employee back to the dashboard.
- Confirmed successful attendance submission.
- Improved the overall user experience.



12. Attendance History

12.1 Attendance History Module

- Created an Attendance History page.
- Retrieved attendance records from the SQLite database.
- Displayed attendance information in a table.
- Included the attendance date, time, and status.

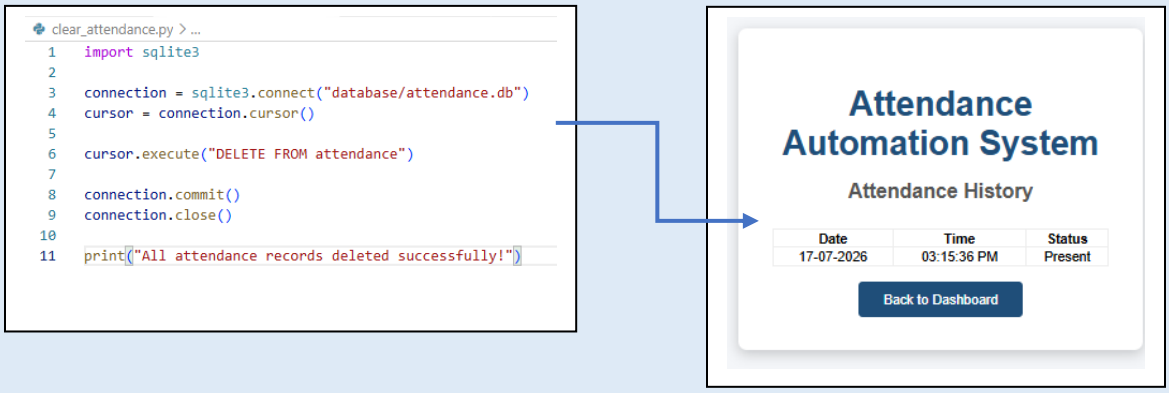
12.2 Dashboard Navigation Update

- Connected the Attendance History button to the history page.
- Enabled employees to view attendance records directly from the dashboard.
- Improved navigation within the application.
- Simplified access to attendance information.

13. Prevent Duplicate Attendance

13.1 Implementing Duplicate Attendance Prevention

- Added a duplicate attendance validation mechanism.
- Checked whether the employee had already marked attendance for the current date.
- Prevented duplicate attendance records from being inserted into the database.
- Ensured that each employee could mark attendance only once per day.

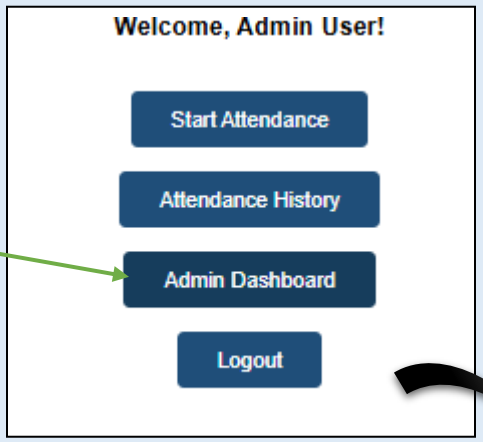


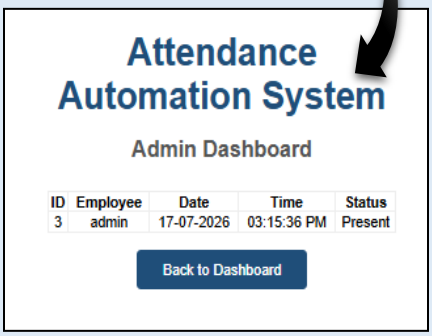
14. Admin Dashboard

14.1 Creating the Admin Dashboard

- Created an Admin Dashboard page.
- Displayed attendance records in a tabular format.
- Included employee username, date, time, and attendance status.
- Provided administrators with a centralized attendance management interface.

```python
@app.route('/admin-dashboard')
def admin_dashboard():

    if 'username' not in session:
        return redirect(url_for('login'))

    connection = sqlite3.connect("database/attendance.db")
    cursor = connection.cursor()

    cursor.execute("""
        SELECT id,
               employee_username,
               attendance_date,
               attendance_time,
               status
        FROM attendance
        ORDER BY id DESC
    """)

    records = cursor.fetchall()

    connection.close()

    return render_template(
        "admin_dashboard.html",
        records=records
    )
```





14.2 Connecting the Admin Dashboard with the Database

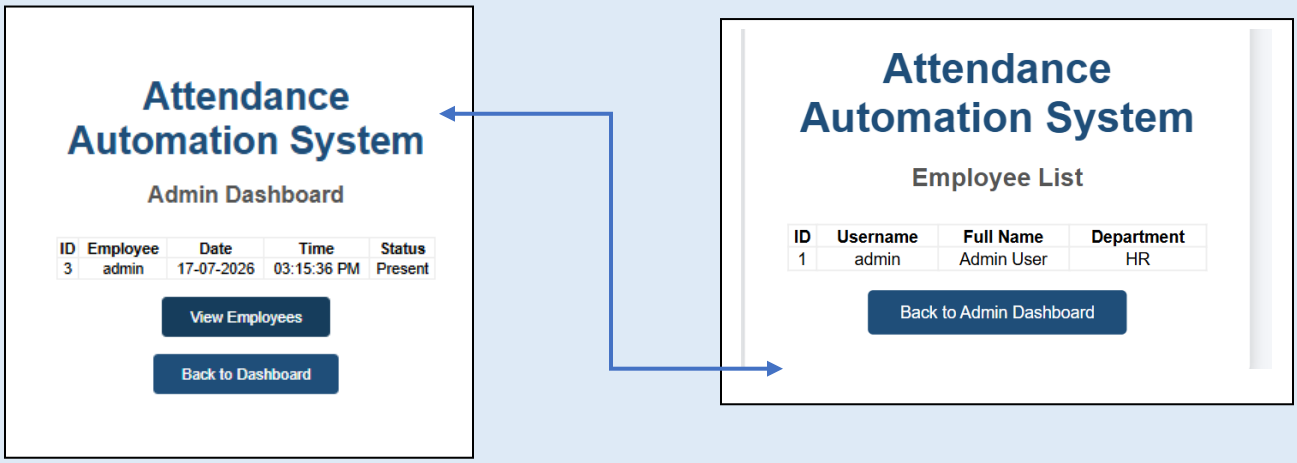
- Retrieved attendance records from the SQLite database.
- Displayed database records dynamically using Flask.
- Organized attendance information in a structured table.
- Enabled administrators to monitor employee attendance efficiently.

15. Employee Management

15.1 Employee Management Module

- Developed an Employee Management page.
- Retrieved employee information from the SQLite database.
- Displayed employee ID, username, full name, and department.
- Provided administrators with a centralized employee management interface.

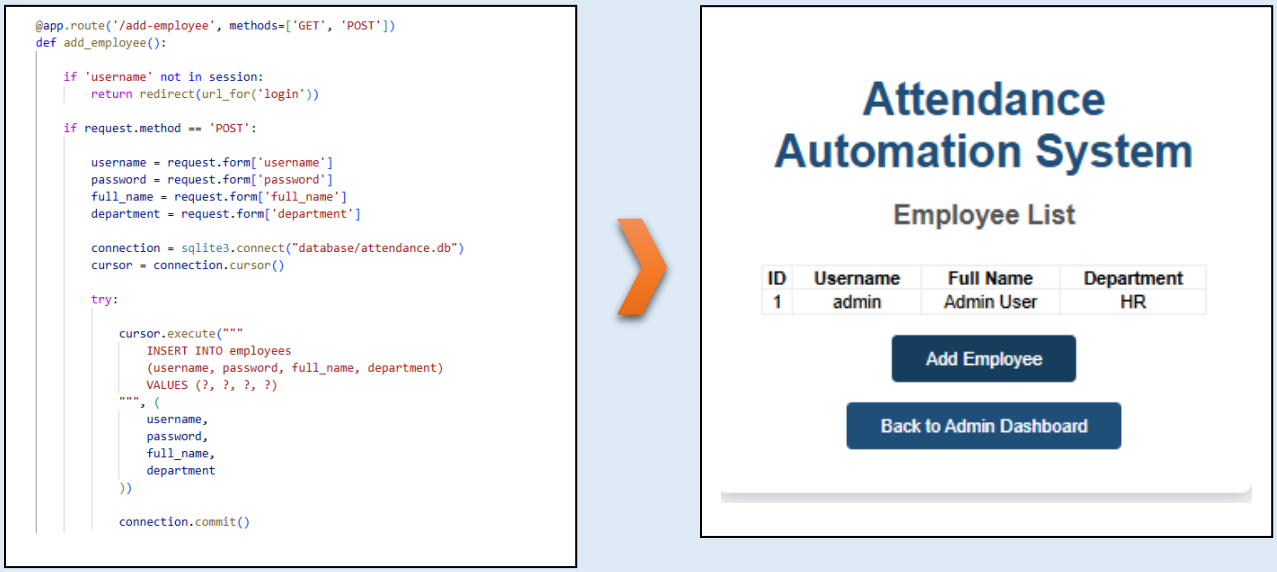
```python
@app.route('/employees')
def employees():

    if 'username' not in session:
        return redirect(url_for('login'))

    connection = sqlite3.connect("database/attendance.db")
    cursor = connection.cursor()

    cursor.execute("""
        SELECT id,
               username,
               full_name,
               department
        FROM employees
        ORDER BY id
    """)

    employees = cursor.fetchall()

    connection.close()

    return render_template(
        "employees.html",
        employees=employees
    )
```



16. Add New Employee

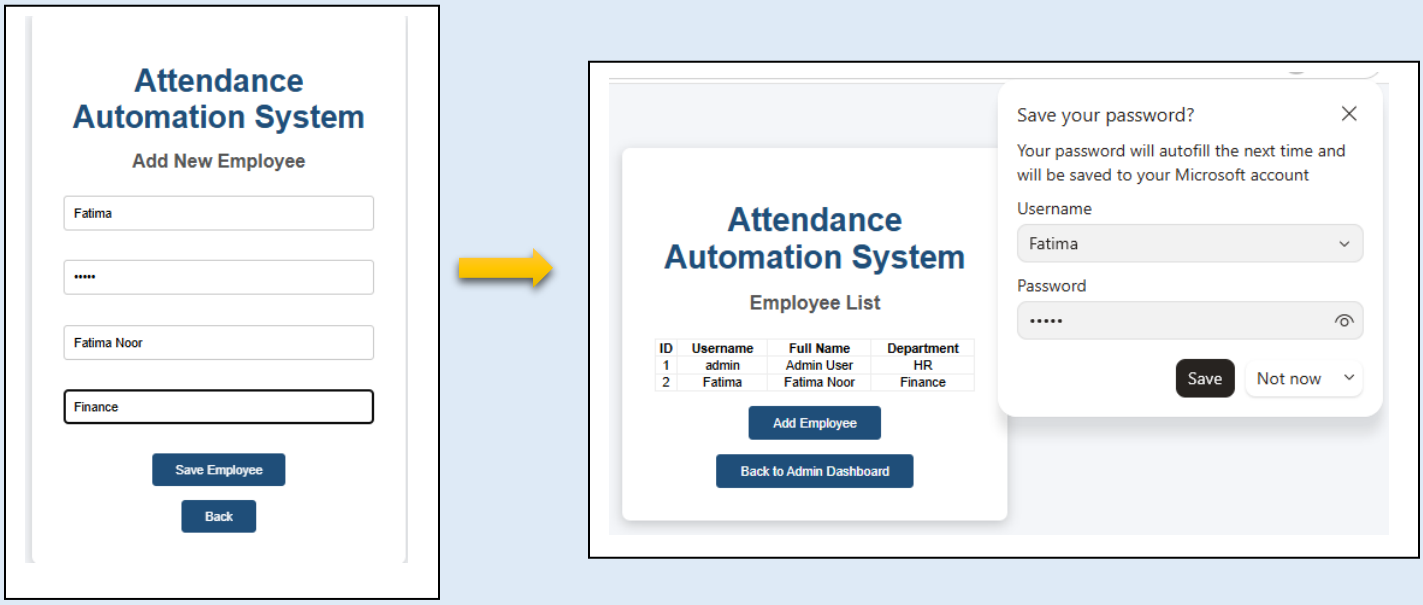
16.1 Add Employee Interface

- Created an HTML form for employee registration.
- Collected the employee's username, password, full name, and department.
- Designed a simple and user-friendly interface.
- Prepared the form for database integration.



16.2 Employee Registration

- Connected the registration form with Flask and SQLite.
- Inserted employee information into the Employees table.
- Checked for duplicate usernames before registration.
- Stored new employee records successfully in the database.



17. Add a Role Column

17.1 Adding User Roles

- Added a **role** column to the Employees table.
- Assigned the **admin** role to administrators.
- Assigned the **employee** role to regular users.
- Prepared the system for role-based authentication.

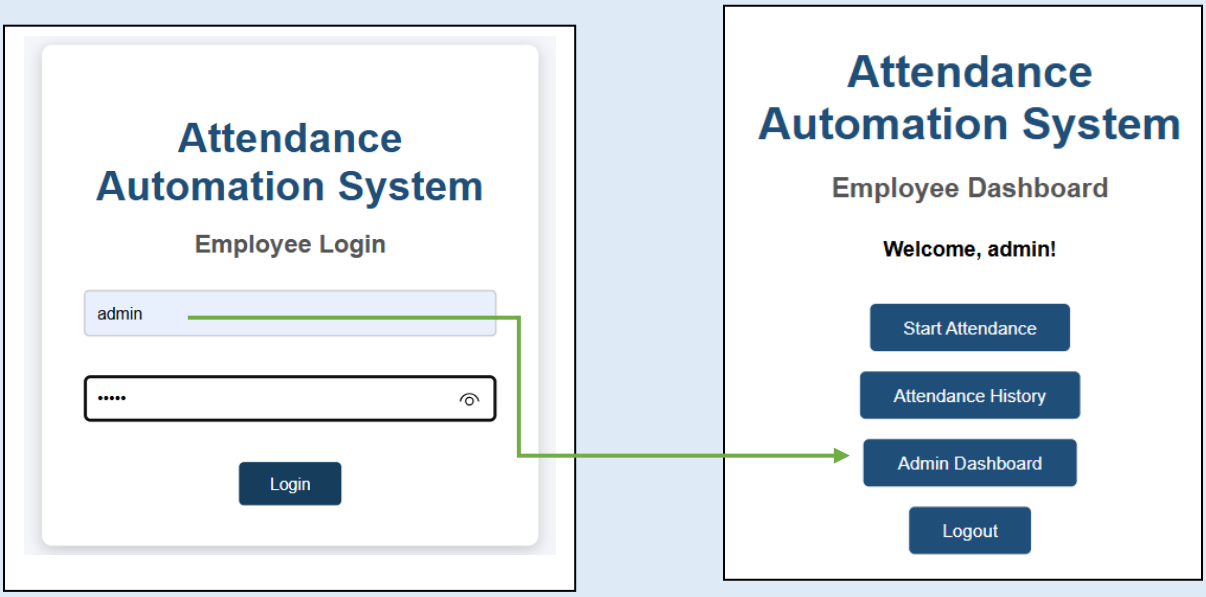
	id	username	password	full_name	departm...	role
1	1	admin	12345	Admin User	HR	a.
2	2	Fatima	23452	Fatima Noor	Finance	e.

18. User Roles & Access Control

18.1 Implementing Role-Based Access Control

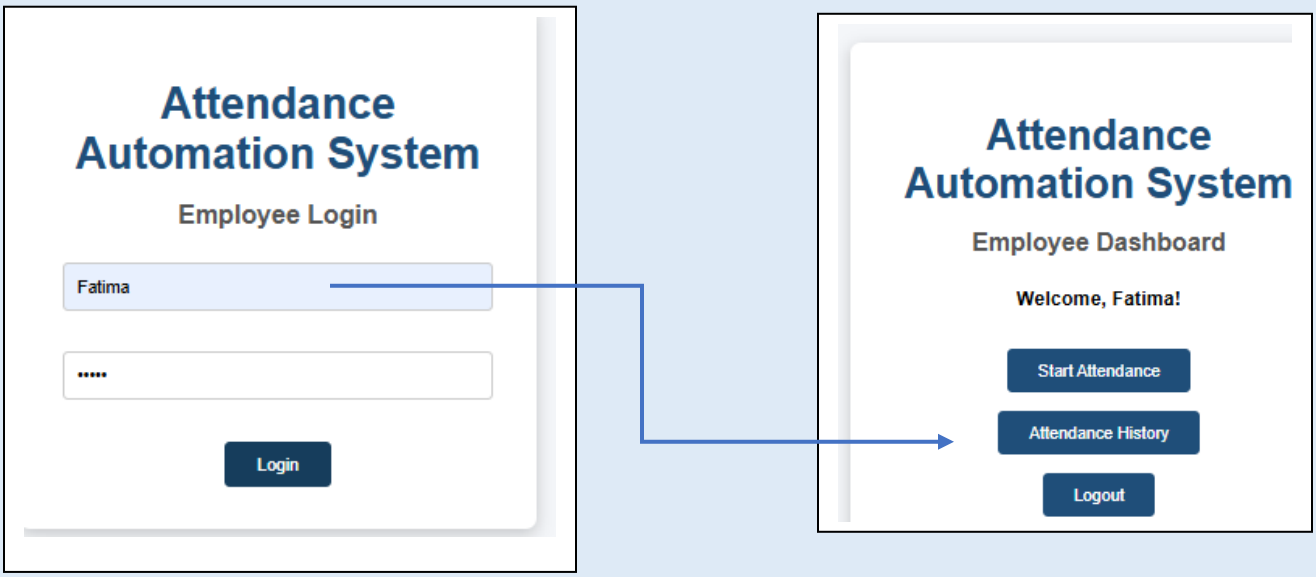
- Retrieved the user's role during login.
- Stored the role information in the Flask session.

- Granted different system permissions based on the assigned role.
- Improved application security through role-based authorization.



18.2 Restricting Unauthorized Access

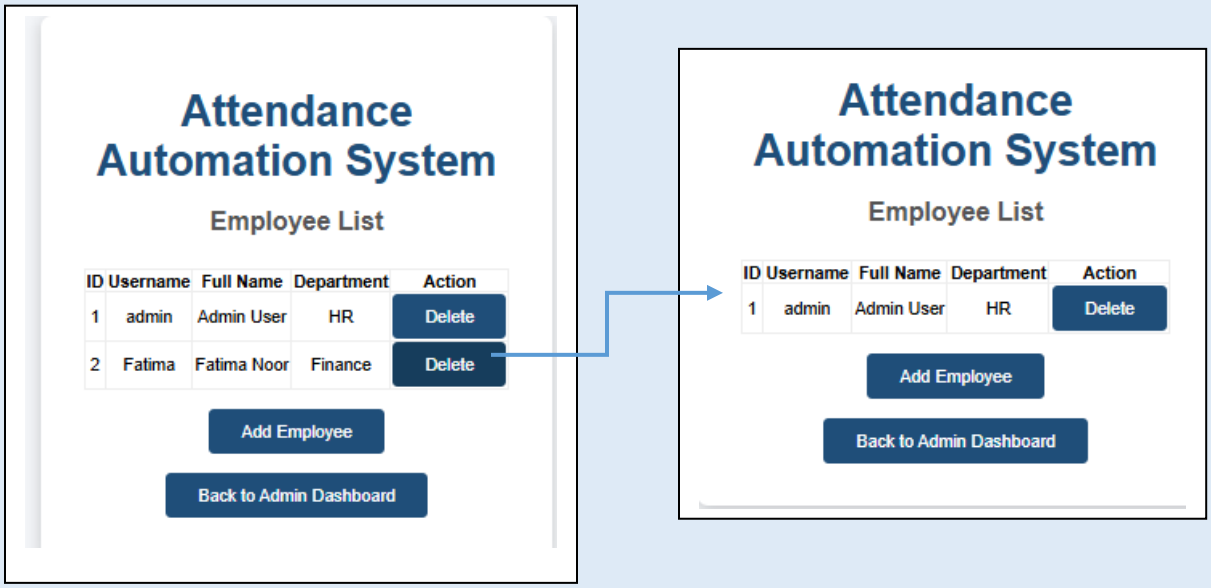
- Allowed administrators to access management features.
- Restricted employees from accessing administrator pages.
- Protected admin routes using session validation.
- Prevented unauthorized access through direct URL navigation.



19. Delete Employee Functionality

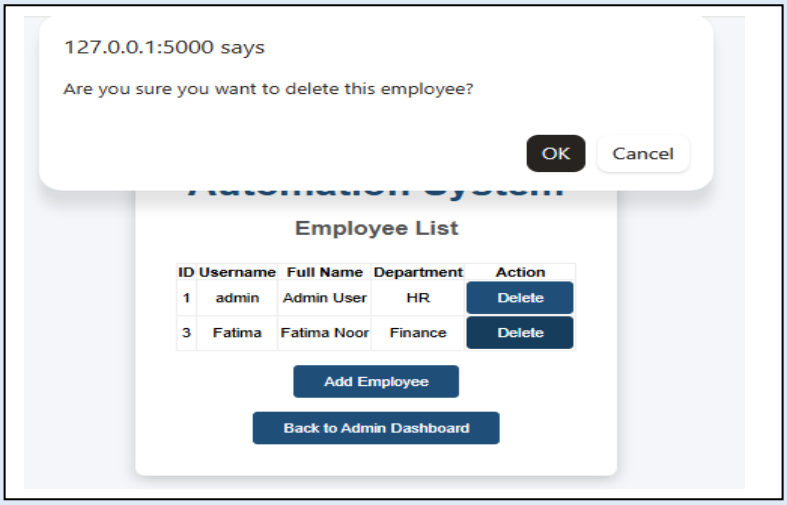
19.1 Deleting Employee Records

- Added a Delete button beside each employee record.
- Linked each button with the employee's unique ID.
- Allowed administrators to remove employee records.
- Updated the employee list automatically after deletion.



19.2 Delete Confirmation

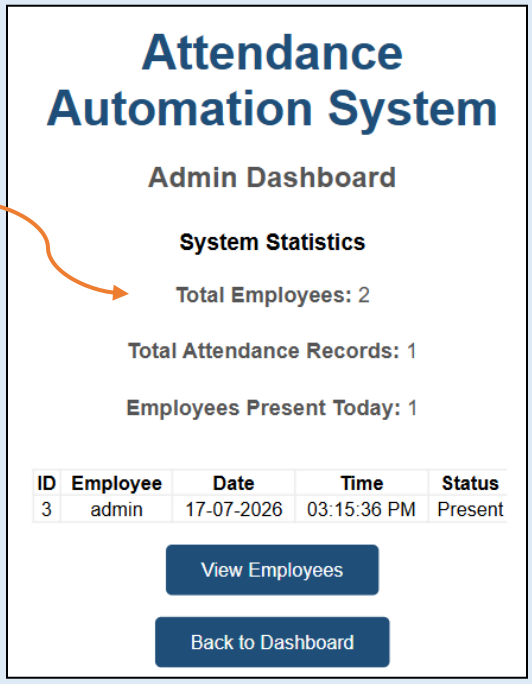
- Added a JavaScript confirmation dialog.
- Requested administrator confirmation before deleting a record.
- Prevented accidental employee deletion.
- Improved system reliability and user safety.



20. Attendance Reports

20.1 Attendance Statistics Dashboard

- Displayed attendance statistics on the Admin Dashboard.
- Showed the total number of employees.
- Displayed total attendance records.
- Displayed employees present and late for the current day.

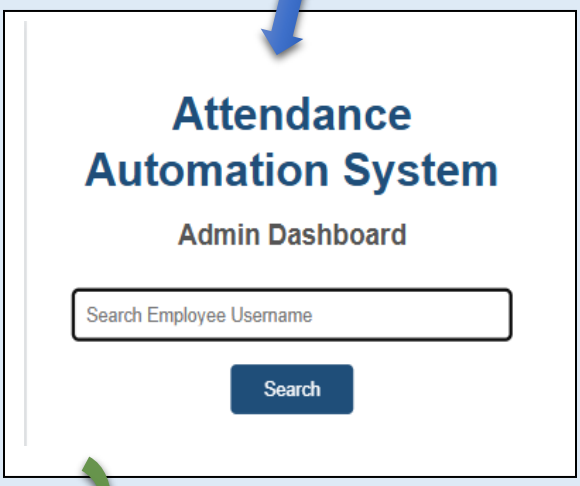


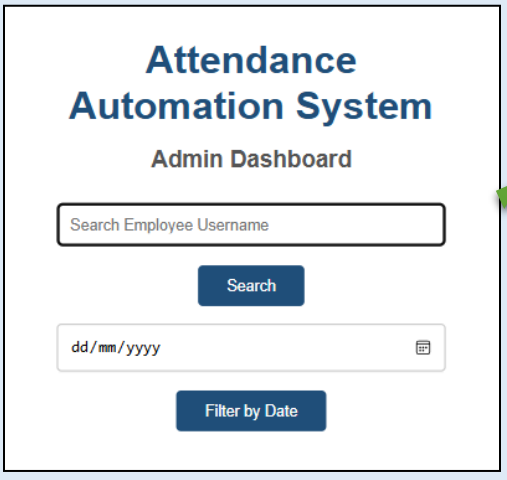
20.2 Search Attendance Records

- Added a search feature for attendance records.
- Allowed administrators to search by employee username.
- Retrieved matching records from the SQLite database.
- Updated the attendance table dynamically.

20.3 Attendance Date Filter

- Added a date filter to the Admin Dashboard.
- Allowed administrators to select a specific attendance date.
- Retrieved matching records using SQL queries.
- Displayed filtered attendance information in the dashboard.

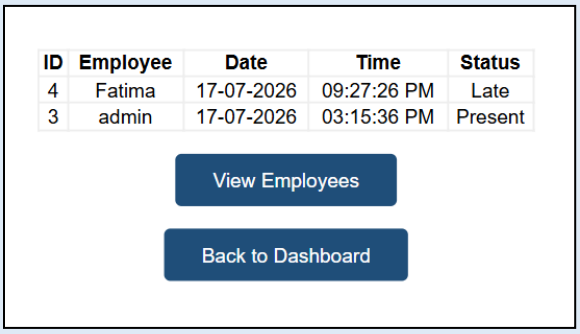




21. Late Attendance Detection

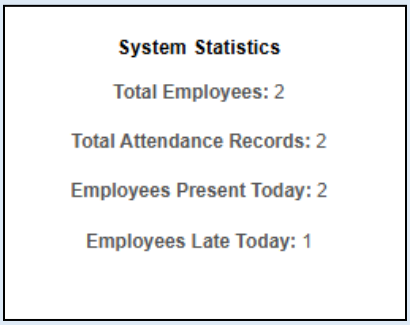
21.1 Implementing Late Attendance Detection

- Added automatic attendance status detection.
- Compared the employee's check-in time with the defined office timing.
- Marked employees as **Present** if they checked in on or before **9:00 AM**.
- Marked employees as **Late** if they checked in after **9:00 AM**.



21.2 Displaying Late Attendance Statistics

- Retrieved late attendance records from the SQLite database.
- Calculated the total number of late employees for the current day.
- Displayed late attendance statistics on the Admin Dashboard.
- Improved attendance monitoring and reporting.

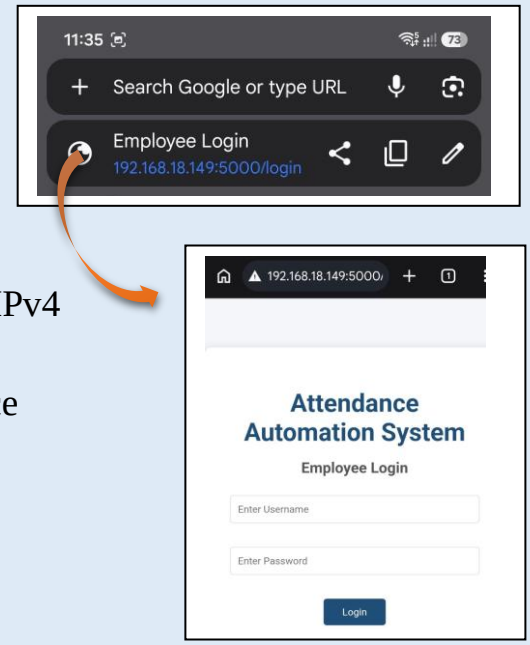


22. Network Access Configuration

22.1 Configuring Network Access

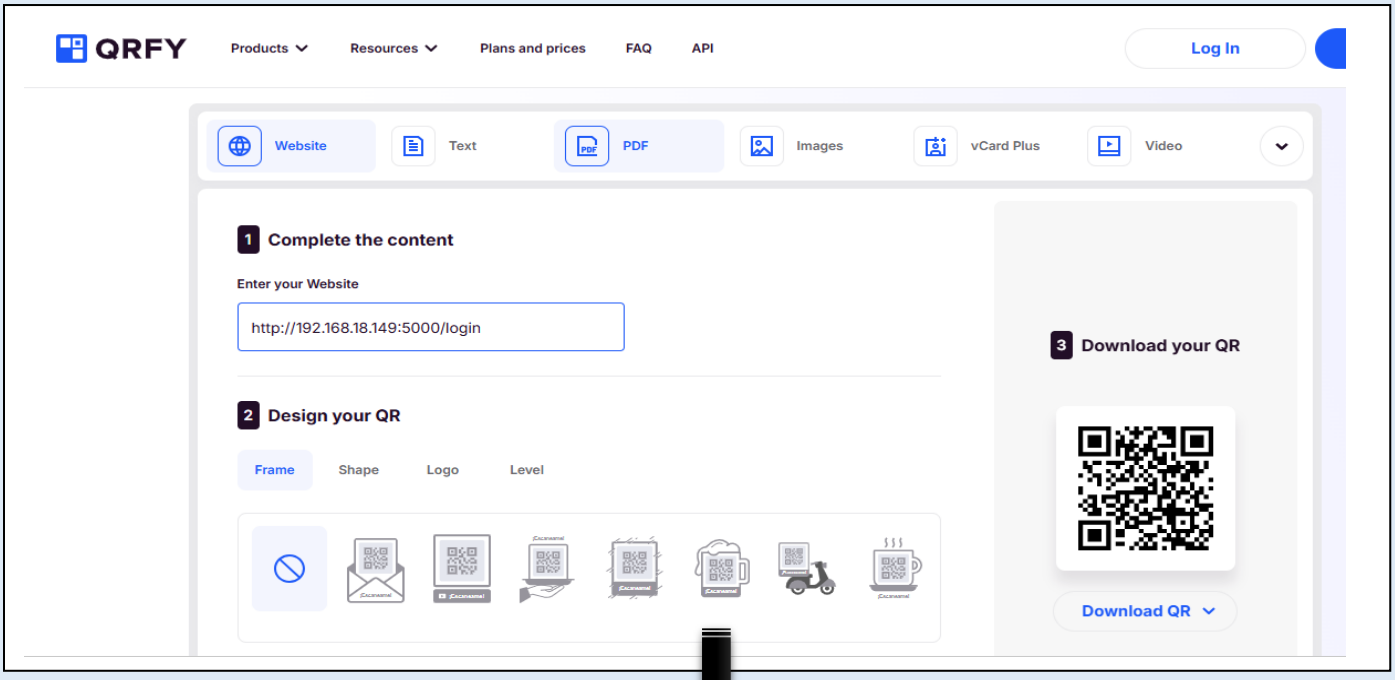
- Configured the Flask server to listen on all network interfaces.
- Changed the server host to **0.0.0.0**.
- Accessed the application using the computer's local IPv4 address.
- Successfully tested the application on a mobile device connected to the same Wi-Fi network.

On Phone:



22.2 Generating the Office QR Code

- Generated a QR code containing the local network URL.
- Allowed employees to access the attendance system by scanning the QR code.
- Eliminated the need to manually enter the website address.
- Improved the convenience of the attendance process.



23. Export Attendance to Excel

23.1 Installing the Excel Library

- Installed the **OpenPyXL** library.
- Imported the required modules into the Flask application.
- Added support for generating Excel reports.
- Prepared the application for Excel file downloads.

```
(venv) PS C:\Users\LENOVO\OneDrive\Desktop\Attendance_Automation_System> pip install openpyxl
Collecting openpyxl
  Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
Collecting et-xmlfile (from openpyxl)
  Downloading et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Downloading openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Installing collected packages: et-xmlfile, openpyxl
Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5
(venv) PS C:\Users\LENOVO\OneDrive\Desktop\Attendance_Automation_System>
```

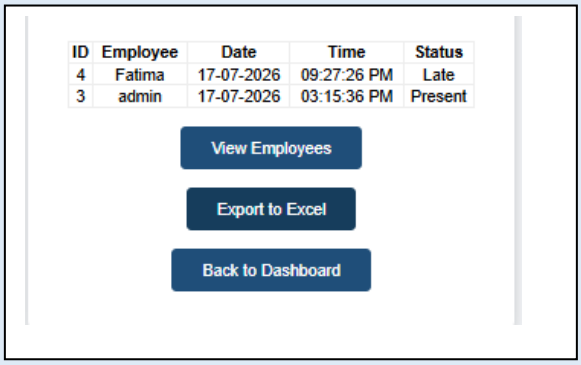
23.2 Generating the Excel Report

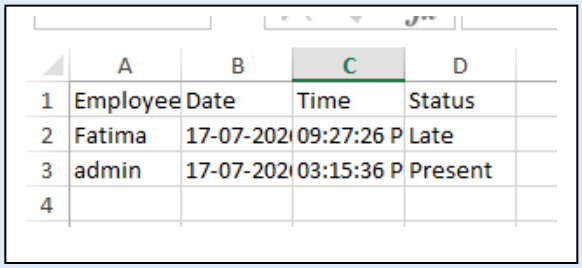
- Created a Flask route to export attendance records.
- Retrieved attendance data from the SQLite database.
- Generated an Excel workbook containing attendance information.
- Automatically downloaded the report when requested.

```python
# ============================
# Export Attendance to Excel
# ============================
@app.route('/export-excel')
def export_excel():

    if 'username' not in session:
        return redirect(url_for('login'))

    if session['role'] != 'admin':
        return redirect(url_for('dashboard'))

    connection = sqlite3.connect("database/attendance.db")
    cursor = connection.cursor()

    cursor.execute("""
        SELECT employee_username,
               attendance_date,
               attendance_time,
               status
        FROM attendance
        ORDER BY id DESC
    """)
```

23.3 Excel Export Interface

- Added an **Export to Excel** button to the Admin Dashboard.
- Connected the button to the Excel export route.
- Allowed administrators to download attendance reports.
- Simplified attendance reporting and record keeping.

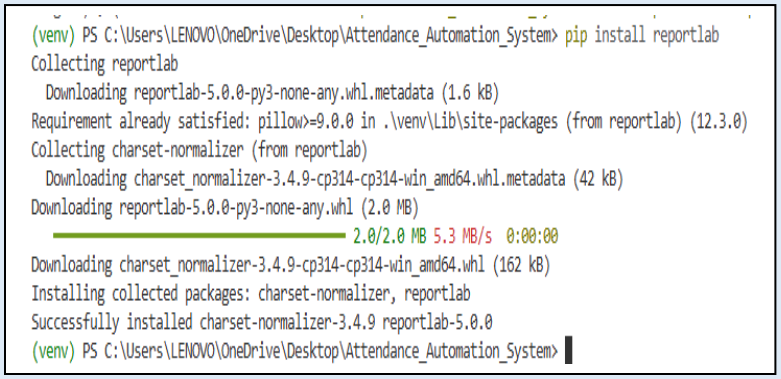




24. Export Attendance to PDF

24.1 Installing the PDF Generation Library

- Installed the **ReportLab** library.
- Integrated ReportLab with the Flask application.
- Prepared the system for PDF report generation.
- Enabled professional report creation.

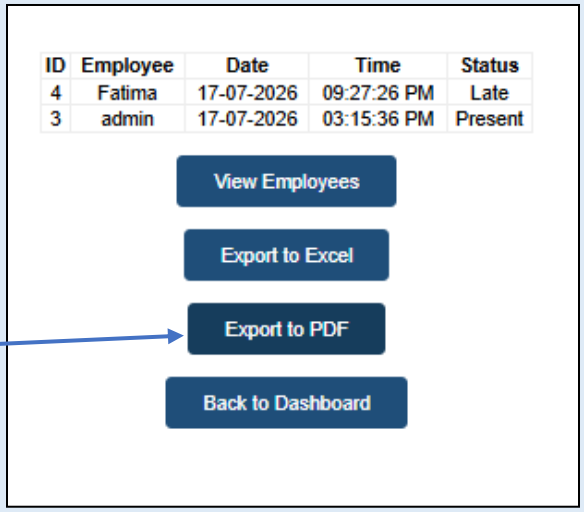


24.2 Importing the PDF Libraries

- Imported the required ReportLab modules.
- Added support for creating PDF documents and tables.
- Applied formatting such as colors, borders, and text alignment.
- Prepared the application for PDF generation.

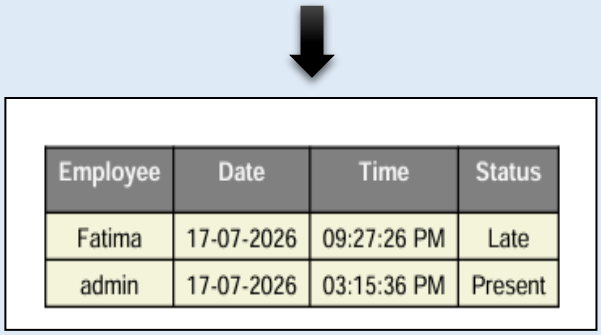
24.3 Generating the PDF Report

- Created a Flask route to generate PDF attendance reports.
- Retrieved attendance records from the SQLite database.
- Displayed attendance information in a formatted table.
- Automatically downloaded the generated PDF report.



24.4 PDF Export Interface

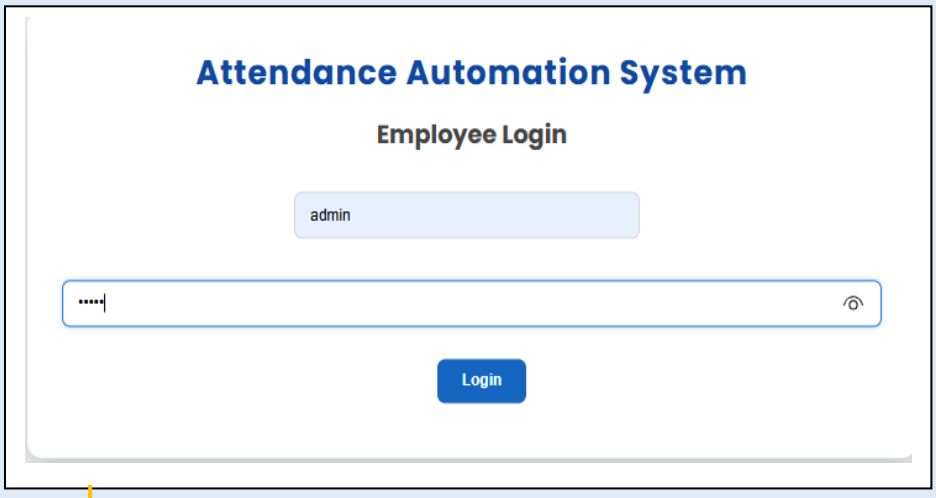
- Added an **Export to PDF** button to the Admin Dashboard.
- Connected the button to the PDF export route.
- Allowed administrators to generate PDF attendance reports.
- Simplified report sharing and record archiving.

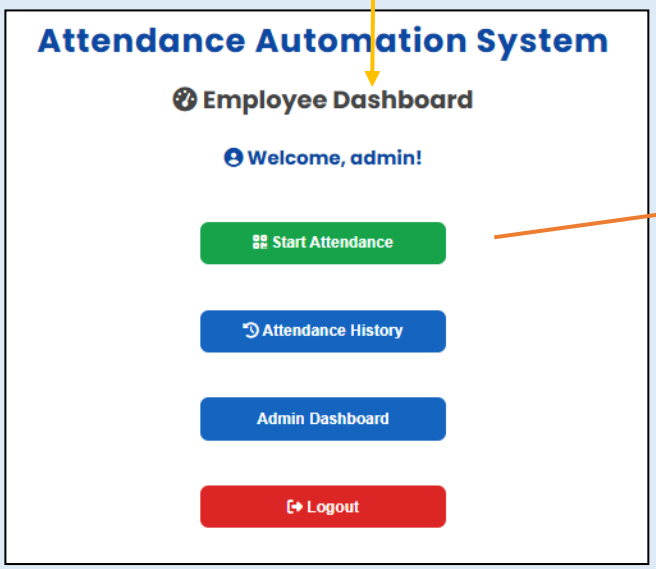


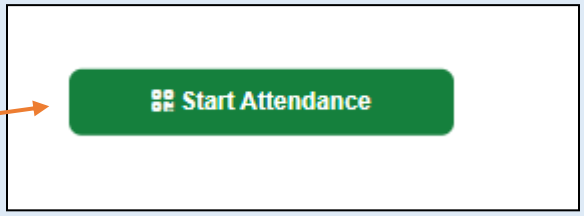
Employee	Date	Time	Status
Fatima	17-07-2026	09:27:26 PM	Late
admin	17-07-2026	03:15:36 PM	Present

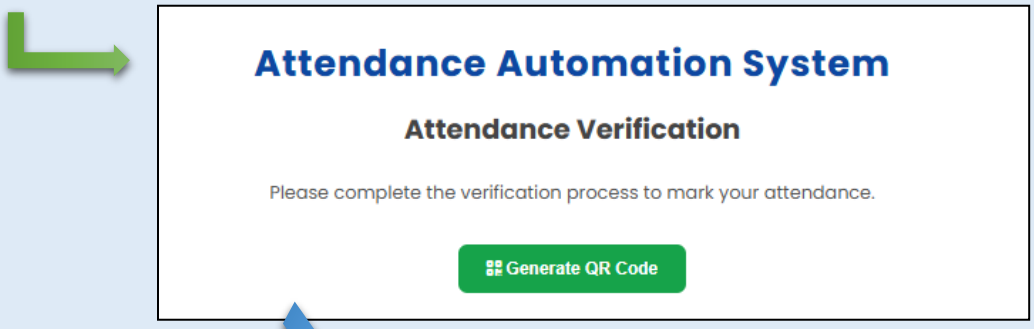
25. Final User Interface

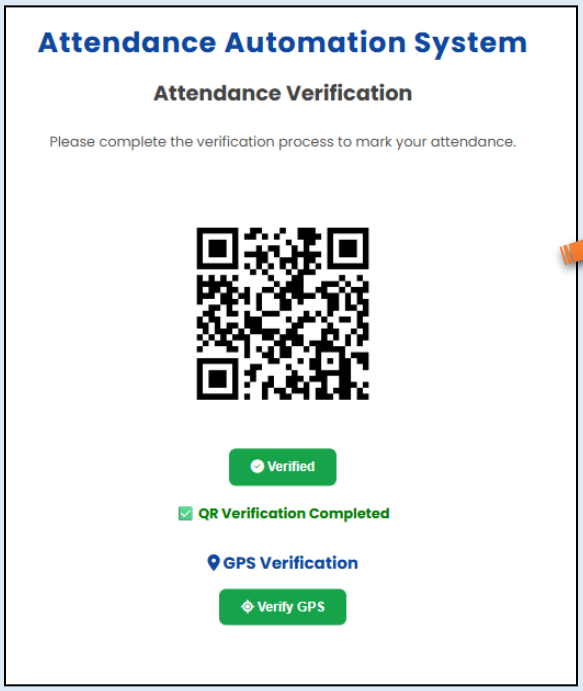
25.1 Logging in as Admin:

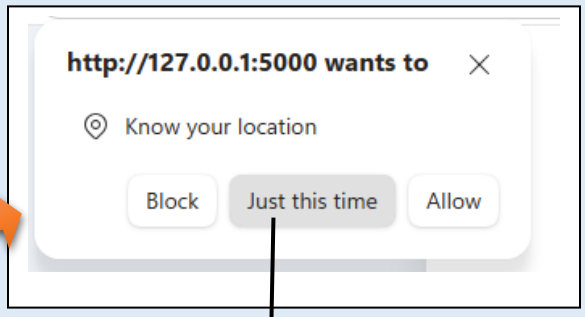


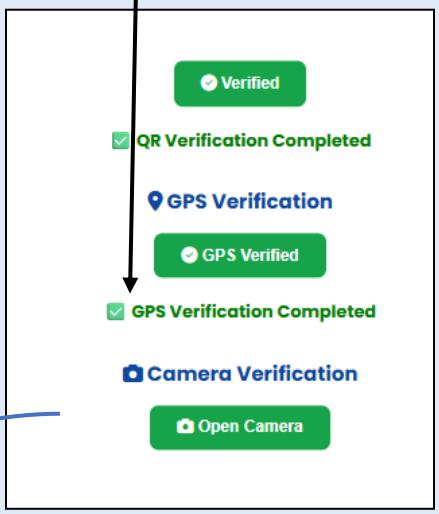


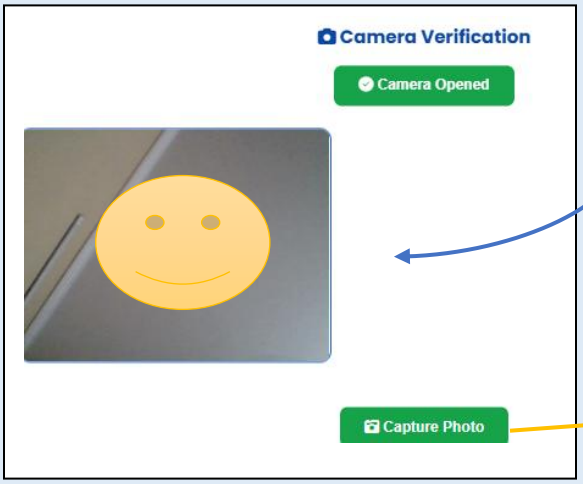


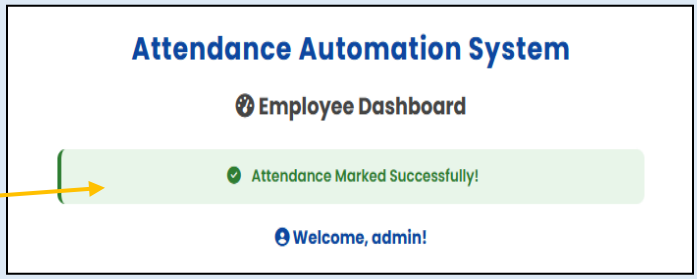

Attendance Automation System
Attendance Verification
Please complete the verification process to mark your attendance.
Generate QR Code

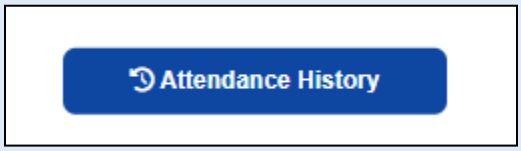

Attendance Automation System
Attendance Verification
Please complete the verification process to mark your attendance.
Verified
QR Verification Completed
GPS Verification
Verify GPS

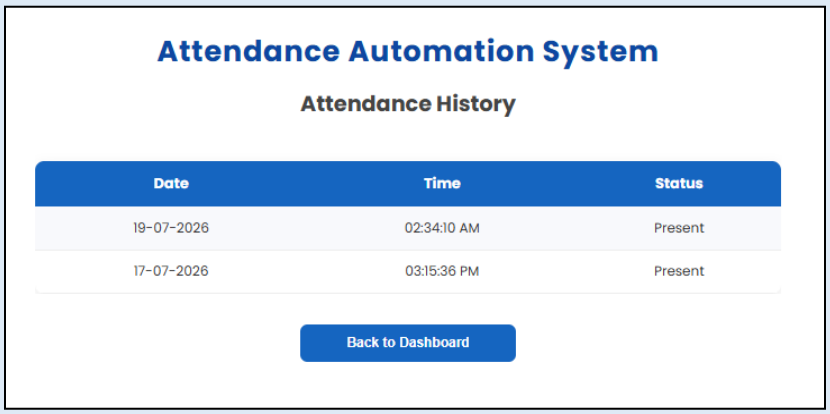

http://127.0.0.1:5000 wants to
Know your location
Block
Just this time
Allow

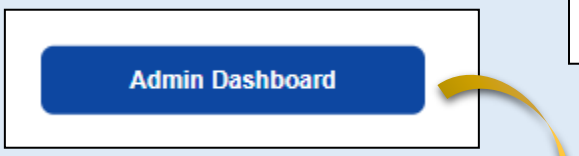

Verified
QR Verification Completed
GPS Verification
GPS Verified
GPS Verification Completed
Camera Verification
Open Camera

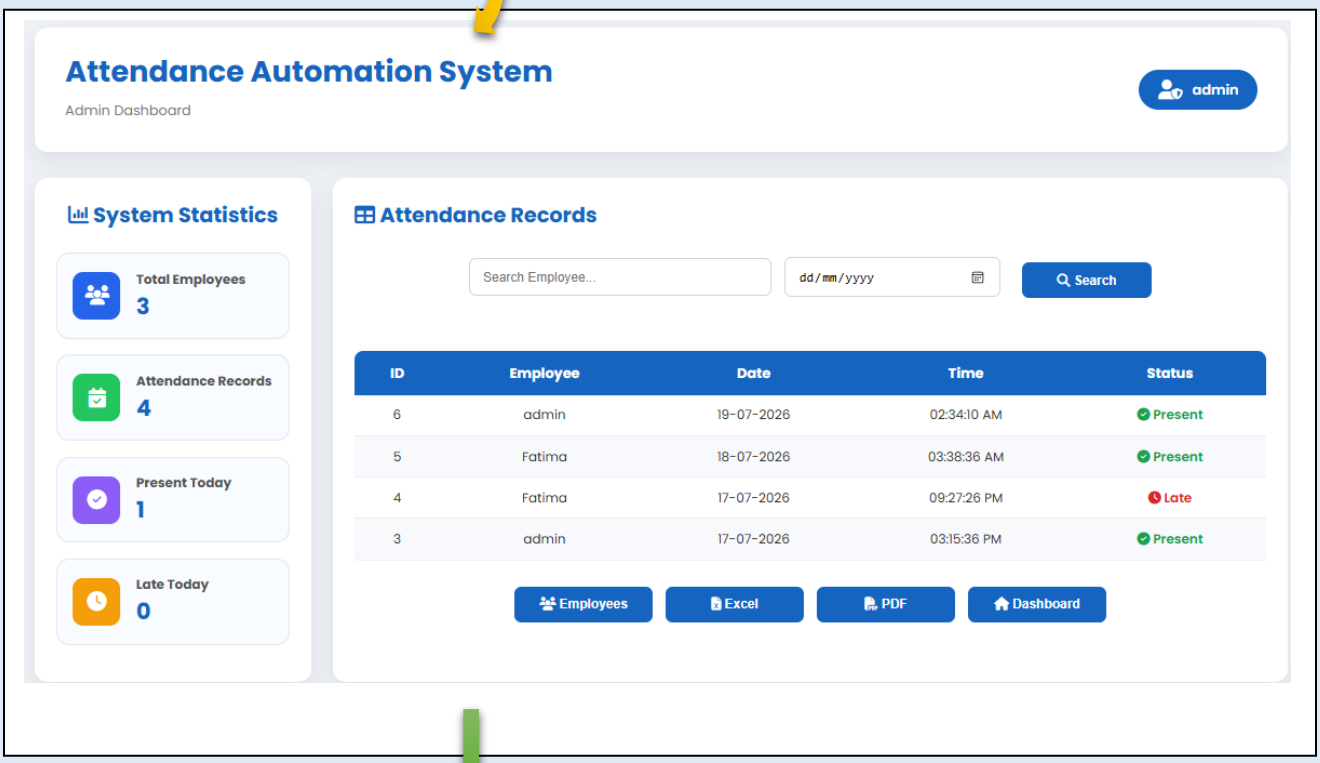

Camera Verification
Camera Opened
Capture Photo


Attendance Automation System
Employee Dashboard
Attendance Marked Successfully!
Welcome, admin!

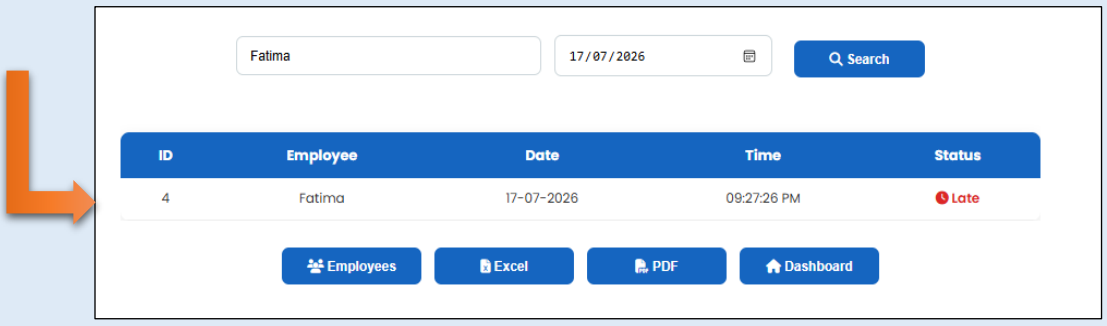

Attendance History

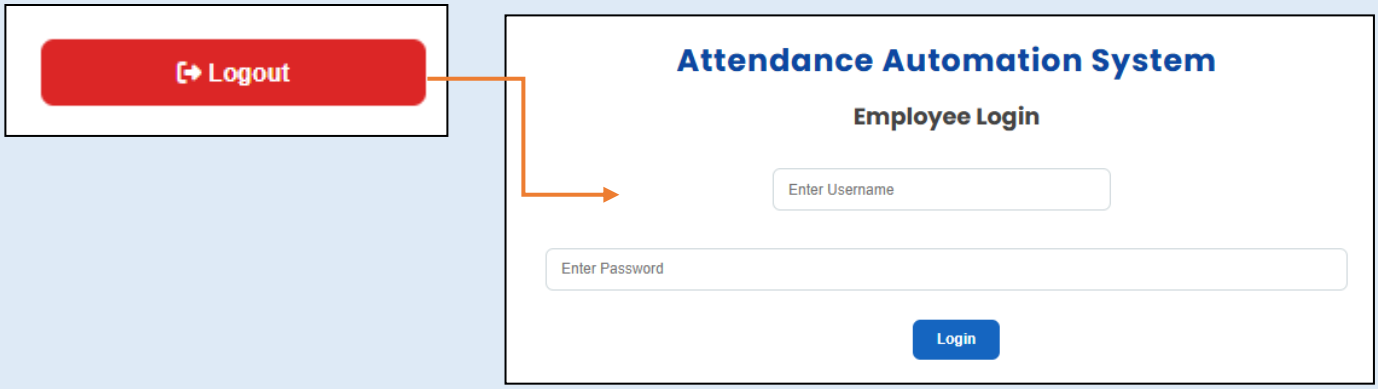

Attendance Automation System
Attendance History
Date
Time
Status
19-07-2026
02:34:10 AM
Present
17-07-2026
03:15:36 PM
Present
Back to Dashboard


Admin Dashboard

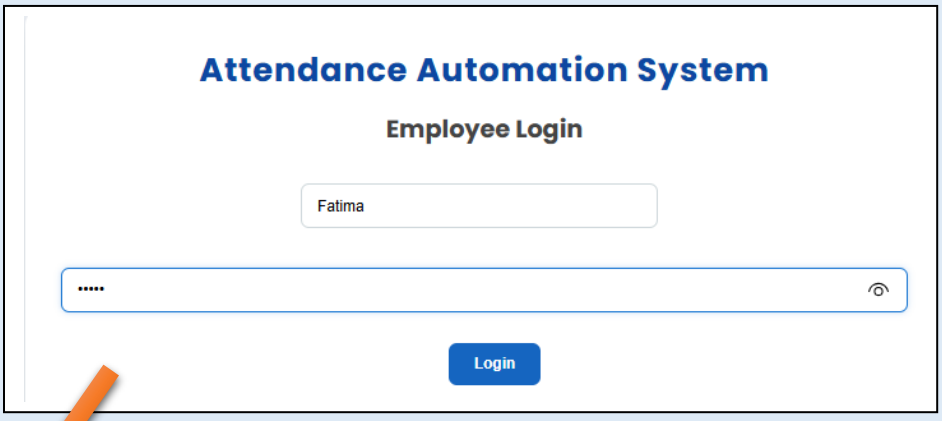

Attendance Automation System
Admin Dashboard
admin
System Statistics
Total Employees
3
Attendance Records
4
Present Today
1
Late Today
0
Attendance Records
Search Employee...
dd/mm/yyyy
Search
ID
Employee
Date
Time
Status
6
admin
19-07-2026
02:34:10 AM
Present
5
Fatima
18-07-2026
03:38:36 AM
Present
4
Fatima
17-07-2026
09:27:26 PM
Late
3
admin
17-07-2026
03:15:36 PM
Present
Employees
Excel
PDF
Dashboard

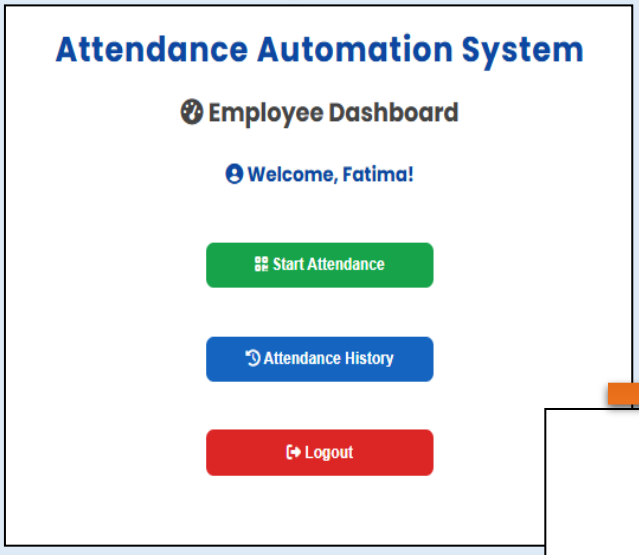

Fatima
17/07/2026
Search

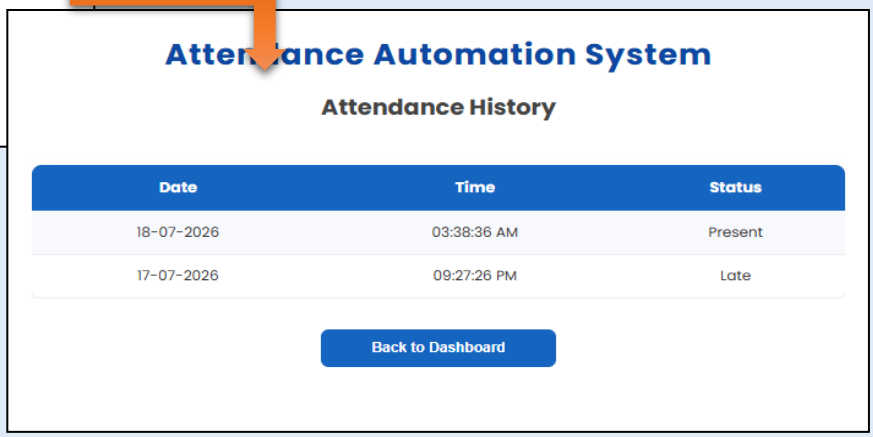

Fatima
17/07/2026
Search
ID
Employee
Date
Time
Status
4
Fatima
17-07-2026
09:27:26 PM
Late
Employees
Excel
PDF
Dashboard



25.2 Logging in as Employee:







26. Conclusion

The Attendance Automation System was successfully designed and developed using **Python, Flask, SQLite, HTML, CSS, and JavaScript**. The project successfully automates the employee attendance process by replacing manual attendance methods with a secure and efficient web-based system.

The system provides secure user authentication, QR code-based attendance access, GPS verification, camera verification, attendance recording, attendance history, and duplicate attendance prevention. An administrator can manage employees, monitor attendance through a centralized dashboard, search and filter records, detect late arrivals, and generate attendance reports in both Excel and PDF formats.

Throughout the project, modern web development concepts such as Flask routing, session management, database integration, role-based access control, and report generation were implemented. The system was also configured for local network access, allowing employees to access the attendance portal using a QR code on the same Wi-Fi network.

Overall, the project meets its objectives by providing a reliable, user-friendly, and secure attendance management solution. It demonstrates the practical application of web development, database management, and software engineering concepts while offering a strong foundation for future enhancements such as facial recognition, cloud database integration, email notifications, and mobile application support.